



Universidade Estadual de Santa Cruz – UESC

**Relatórios de Implementações de Métodos da Disciplina
Análise Numérica**

Relatório de implementações realizadas

por Reinan Lopes Argolo

Disciplina Análise Numérica.

Curso Ciência da Computação

Semestre 2026.1

Professor Gesil Sampaio Amarante II

Ilhéus – BA

2026

Sumário

1	Linguagem Escolhida e Justificativas	6
2	Método da Bissecção	7
2.1	Estratégia de Implementação	7
2.2	Estrutura dos Arquivos de Entrada/Saída	8
2.2.1	Parâmetros de Entrada para a Bissecção	9
2.3	Problemas Teste	9
2.3.1	Problema Teste 1: Tempo de Subida de um Amplificador	9
2.3.2	Problema Teste 2: Ângulo de Lançamento Balístico de um Míssil	11
2.3.3	Problema Teste 3: Análise de Taxas de Juros em Planos de Financiamento	13
2.4	Dificuldades Enfrentadas	15
3	Método da Posição Falsa	17
3.1	Estratégia de Implementação	17
3.2	Estrutura dos Arquivos de Entrada e Saída	18
3.3	Problemas Teste	19
3.3.1	Problema Teste 1: Tempo de Subida de um Amplificador	19
3.3.2	Problema Teste 2: Ângulo de Lançamento Balístico de um Míssil	21
3.3.3	Problema Teste 3: Análise de Taxas de Juros em Planos de Financiamento	23
3.4	Dificuldades Enfrentadas	25
4	Método do Ponto Fixo	26
4.1	Fundamentação Teórica	26
4.2	Estratégia de Implementação	26
4.3	Estrutura dos Arquivos de Entrada e Saída	27
4.4	Problemas Teste	28
4.4.1	Problema Teste 1: Tempo de Subida de um Amplificador	28
4.4.2	Problema Teste 2: Ângulo de Lançamento Balístico de um Míssil	32
4.4.3	Problema Teste 3: Análise de Taxas de Juros em Planos de Financiamento	34
4.5	Dificuldades Enfrentadas	37
5	Método de Newton-Raphson	39
5.1	Fundamentação Teórica	39
5.2	Estratégia de Implementação	39

5.3	Estrutura dos Arquivos de Entrada e Saída	41
5.4	Problemas Teste	41
5.4.1	Problema Teste 1: Tempo de Subida de um Amplificador	41
5.4.2	Problema Teste 2: Ângulo de Lançamento Balístico de um Míssil	43
5.4.3	Problema Teste 3: Análise de Taxas de Juros em Planos de Fi- nanciamento	45
5.5	Dificuldades Enfrentadas	47
6	Método da Secante	48
6.1	Fundamentação Teórica	48
6.2	Estratégia de Implementação	48
6.3	Estrutura dos Arquivos de Entrada e Saída	49
6.4	Problemas Teste	50
6.4.1	Problema Teste 1: Tempo de Subida de um Amplificador	50
6.4.2	Problema Teste 2: Ângulo de Lançamento Balístico de um Míssil	51
6.4.3	Problema Teste 3: Análise Crítica - Falhas Físicas em Planos de Financiamento	52
6.5	Dificuldades Enfrentadas	53
7	Considerações Finais	54

Lista de Figuras

1	Convergência progressiva da magnitude residual durante a execução para $T_{0.1}$	10
2	Comportamento da convergência em escala logarítmica para a busca de α (Problema 3.6). A inclinação reta atesta a taxa de convergência linear do método.	13
3	Gráfico de convergência atestando a contração linear do hipervolume de erro para a determinação da taxa de juros no Plano A.	15
4	Amortecimento do erro relativo e do valor residual para $f_1(T)$. A inclinação contínua e acentuada nos primeiros estágios atesta a rápida convergência local proporcionada pela interpolação linear da secante. .	20
5	Gráfico de convergência da aproximação do ângulo de lançamento α pelo Método da Posição Falsa. A queda abrupta nas primeiras iterações valida o alto desempenho da interpolação linear frente à topologia da função analisada.	22
6	Lentidão inerente ao Método da Posição Falsa na avaliação dos planos. A suavidade da curva de decaimento do erro denota a penalização imposta pela alta convexidade das funções, exigindo quase 100 iterações para o Plano B.	24
7	Comparativo analítico das taxas de convergência para $T_{0.1}$. A acentuada inclinação da curva de $g_3(T)$ comprova a superioridade matemática da modelagem logarítmica sobre o amortecimento empírico linear de $g_7(T)$. .	30
8	Gráfico de decaimento do erro iterativo demonstrando a convergência linear. A inclinação constante do decréscimo em escala semi-logarítmica é o traço característico governado pela constante de Lipschitz associada à derivada $g'(T)$	32
9	Decaimento logarítmico do erro iterativo para o ângulo do míssil. A inclinação contínua da reta evidencia o estrito cumprimento da condição de Lipschitz pela função $g_3(\alpha)$, em forte contraste com as instabilidades observadas nos modelos descartados.	34
10	Comportamento da convergência para a determinação de J_A e J_B . O decréscimo contínuo e altamente inclinado em escala logarítmica atesta o sucesso da técnica de amortecimento na estabilização de derivadas elevadas em polinômios de alto grau.	36

11	Gráfico representativo do amortecimento logarítmico para $f_1(T)$. Nota-se que, distintamente dos métodos lineares (cujas retas de erro possuem declividade constante), a curva aqui sofre uma inflexão convexa para baixo, revelando a aceleração de segunda ordem onde os algarismos significativos multiplicam-se a cada passo.	43
12	Gráfico de convergência atestando a extrema eficiência iterativa de Newton-Raphson na avaliação trigonométrica. O alcance do patamar de 10^{-6} em apenas 3 passos reflete a assertividade da aproximação linear na vizinhança correta da raiz.	45
13	Gráfico comparativo da convergência para ambos os planos de financiamento. A trajetória essencialmente vertical em escala semi-logarítmica, extinguindo o erro em 3 iterações, destoa substancialmente do arrasto sublinear outrora documentado pelo Método da Posição Falsa, sagrando Newton-Raphson como a ferramenta primária para este perfil topológico.	46
14	Curva de erro demonstrando a taxa de convergência associada à Razão Áurea (≈ 1.618). Nota-se o decaimento brutal nas iterações 4 e 5, evidenciando o domínio da bacia de atração.	51

Lista de Tabelas

1	Parâmetros de configuração para o Método da Bissecção.	9
2	Histórico iterativo do Método da Bissecção para a busca de $T_{0.1}$ na função $f_1(T)$	11
3	Histórico iterativo do Método da Bissecção para a função do ângulo de lançamento balístico $f(\alpha)$	12
4	Histórico iterativo do Método da Bissecção para determinação da taxa do Plano A ($f_1(x)$).	14
5	Histórico iterativo do Método da Posição Falsa para a busca de $T_{0.1}$ na função $f_1(T)$	19
6	Histórico iterativo do Método da Posição Falsa para o modelo balístico $f(\alpha)$	21
7	Histórico iterativo do Método da Posição Falsa para determinação da taxa do Plano A ($f_1(x)$).	23
8	Histórico iterativo da função $g_3(T)$ (Isolamento Logarítmico) para a busca de $T_{0.1}$	29
9	Histórico iterativo do Método do Ponto Fixo para a busca de $T_{0.1}$ usando $g_1(T)$ ($T_0 = 0.5$).	31
10	Histórico iterativo do Método do Ponto Fixo para o ângulo de lançamento utilizando a modelagem $g_3(\alpha)$	33
11	Histórico iterativo da função de relaxação $g_1(x)$ ($\lambda = 0.05$) para a taxa do Plano A.	35
12	Histórico iterativo do Método de Newton-Raphson para a busca de $T_{0.1}$ na função $f_1(T)$	42
13	Histórico iterativo do Método de Newton-Raphson para o cálculo do ângulo α	44
14	Histórico iterativo do Método de Newton-Raphson para o polinômio de grau 10 (Plano A).	46
15	Histórico iterativo do Método da Secante para a busca de $T_{0.1}$	50
16	Histórico iterativo para o cálculo de α pelo Método da Secante.	51
17	Convergência anômala (raiz trivial) do Método da Secante para o Plano A.	52

1 Linguagem Escolhida e Justificativas

Para o desenvolvimento dos algoritmos e a execução das rotinas computacionais propostas neste relatório de métodos numéricos, optou-se pela utilização da linguagem de programação Python. A escolha dessa ferramenta justifica-se, pela sua expressiva adoção na comunidade científica e pela vasta disponibilidade de bibliotecas para manipulação matemática e computação científica [3].

Entre as principais extensões que fundamentam essa decisão, destaca-se a biblioteca *NumPy*, a qual fornece suporte nativo e eficiente para o processamento de arranjos e matrizes multidimensionais. Essa biblioteca implementa operações de álgebra linear com rotinas de baixo nível, garantindo o alto desempenho exigido nos cálculos numéricos iterativos [1]. Em complemento, o uso do ecossistema *SciPy* oferece um conjunto de algoritmos fundamentais para interpolação, integração numérica, otimização e resolução de sistemas lineares e não lineares [4].

Ademais, Python caracteriza-se por uma sintaxe de alto nível e semântica clara, o que favorece a legibilidade do código e permite a prototipagem rápida de abstrações matemáticas [2]. Dessa forma, a redução da complexidade de implementação viabiliza um foco maior na análise de convergência, na estabilidade dos métodos e na mitigação de erros de truncamento e arredondamento, requisitos indispensáveis para a consistência deste trabalho acadêmico.

2 Método da Bissecção

O Método da Bissecção é uma abordagem numérica clássica e incondicionalmente convergente para a determinação de raízes reais de equações não lineares. Fundamentado no Teorema de Bolzano, o método opera mediante a divisão sucessiva de um intervalo fechado $[a, b]$, no qual a função contínua $f(x)$ apresenta sinais opostos em seus extremos ($f(a) \cdot f(b) < 0$). Embora possua uma taxa de convergência linear — exigindo um número razoável de iterações —, sua robustez o torna ideal para isolamento inicial de raízes ou resolução de problemas onde a estabilidade é prioritária em relação à velocidade computacional.

2.1 Estratégia de Implementação

A concepção computacional do algoritmo foi estruturada na linguagem Python, priorizando a estabilidade numérica e a modularidade do código. A implementação principal encontra-se encapsulada no módulo `src/metodos/zeros_de_funcao.py`.

Código 1: Código de implementação do Método da Bissecção

```
1 def bisseccao(funcao, a, b, tol=1e-6, max_iter=100):
2     historico = []
3
4     for i in range(max_iter):
5         c = (a + b)/2
6         fc = funcao(c)
7         denominador = a + b
8
9         if denominador != 0:
10             er = abs((a - b) / denominador)
11         else:
12             er = abs(b - a)
13
14         historico.append({
15             'iter': i, 'a': a, 'b': b, 'c': c,
16             'f(c)': fc, 'erro': er
17         })
18
19         if fc == 0 or er < tol:
20             return c, i, historico
21
22         if utils.teorema_bolzano(funcao, a, c):
23             b = c
```

```

24         else:
25             a = c
26
27     return c, max_iter, historico

```

Para a formulação matemática do erro iterativo e visando superar limitações do ambiente de programação, estabeleceu-se uma métrica adaptativa. Originalmente, o erro relativo ponderado pela amplitude do intervalo é definido como $\epsilon_{rel} = \left| \frac{a_k - b_k}{a_k + b_k} \right|$. Contudo, constatou-se que intervalos simétricos em relação à origem ($a_k = -b_k$) induziriam o interpretador Python a lançar uma exceção de divisão por zero (`ZeroDivisionError`). Para mitigar este risco, implementou-se um fluxo condicional: caso o denominador seja nulo, a métrica transita automaticamente para o erro absoluto $\epsilon_{abs} = |b_k - a_k|$.

Os critérios de parada foram definidos para garantir o término do laço em tempo finito, compreendendo:

1. **Raiz exata:** Interrupção imediata caso a avaliação funcional no ponto médio resulte exatamente em zero ($f(c_k) = 0$), limitando processamento desnecessário por flutuação de ponto flutuante.
2. **Tolerância (ϵ_{tol}):** O método encerra com sucesso caso a métrica de erro calculada seja estritamente menor que a tolerância estipulada.
3. **Esforço computacional ($\max_iter$):** Uma trava de segurança máxima de iterações para abortar o processo caso a convergência não seja alcançada no limite imposto.

2.2 Estrutura dos Arquivos de Entrada/Saída

Visando aderência aos princípios de responsabilidade única e reprodutibilidade científica, a persistência de dados foi dissociada da lógica matemática. Os diretórios foram organizados hierarquicamente:

- **Dados de Entrada (`dados/entradas/`):** Estruturados em arquivos de texto simples (`.txt`). Foram escolhidos por permitirem fácil edição manual dos parâmetros de controle (limites a e b , tolerância e limite de iterações) sem necessidade de reconfiguração no código-fonte.
- **Dados de Saída (`dados/saida/<questao>/`):** Os resultados iterativos foram consolidados em arquivos de valores separados por vírgula (`.csv`). A escolha do formato CSV justifica-se por sua interoperabilidade universal, permitindo a ingestão direta por bibliotecas de análise de dados (como Pandas) e por pacotes

de renderização gráfica em LaTeX (como TikZ/pgfplots), viabilizando análises quantitativas automatizadas e plots de convergência nativos no documento final.

2.2.1 Parâmetros de Entrada para a Bissecção

Os arquivos de entrada definem os hiperparâmetros críticos para cada execução do método. A Tabela 1 sumariza os parâmetros utilizados nas questões propostas:

Tabela 1: Parâmetros de configuração para o Método da Bissecção.

Questão	Função	Intervalo [a, b]	Tolerância	Máx Iter
3.3	$f_1(T)$	[1.0, 2.0]	1×10^{-5}	100
3.3	$f_2(T)$	[5.0, 9.0]	1×10^{-5}	100
3.6	$f(x)$	[30.0, 60.0]	1×10^{-4}	100

Observa-se que a tolerância de 10^{-5} na Questão 3.3 exigirá um número maior de iterações comparativamente à tolerância mais relaxada de 10^{-4} na Questão 3.6. O limite máximo de 100 iterações constitui uma salvaguarda conservadora, garantindo que o algoritmo não incorra em consumo excessivo de recursos computacionais.

2.3 Problemas Teste

A verificação e validação da implementação computacional foram conduzidas por meio de problemas aplicados extraídos da literatura de referência de Cálculo Numérico (Franco, Neide). O objetivo desta seção é atestar o funcionamento, a eficiência e a validade matemática das soluções encontradas.

2.3.1 Problema Teste 1: Tempo de Subida de um Amplificador

O primeiro problema propõe a determinação do tempo de subida de um amplificador eletrônico, compreendido como o intervalo temporal necessário para que a resposta $g(T)$ avance de 10% a 90% de seu valor assintótico, sendo a função normalizada expressa por:

$$g(T) = 1 - \left(1 + T + \frac{T^2}{2}\right) e^{-T} \quad (1)$$

O cálculo do tempo de subida, $\Delta T = T_{0.9} - T_{0.1}$, exige a determinação das raízes

das funções auxiliares:

$$f_1(T) = 0.9 - \left(1 + T + \frac{T^2}{2}\right) e^{-T} = 0 \quad (2)$$

$$f_2(T) = 0.1 - \left(1 + T + \frac{T^2}{2}\right) e^{-T} = 0 \quad (3)$$

O algoritmo processou $f_1(T)$ com intervalo inicial $[1.0, 2.0]$ e tolerância 10^{-5} . A Tabela 2 exibe a progressão numérica.

A convergência foi atingida na 16ª iteração, com o erro caindo abaixo da tolerância ($\epsilon_{16} \approx 6.92 \times 10^{-6}$), retornando $T_{0.1} \approx 1.102058$. A Figura 1 ilustra este amortecimento da magnitude funcional, que demonstra a redução progressiva assintótica característica da contração de intervalo.

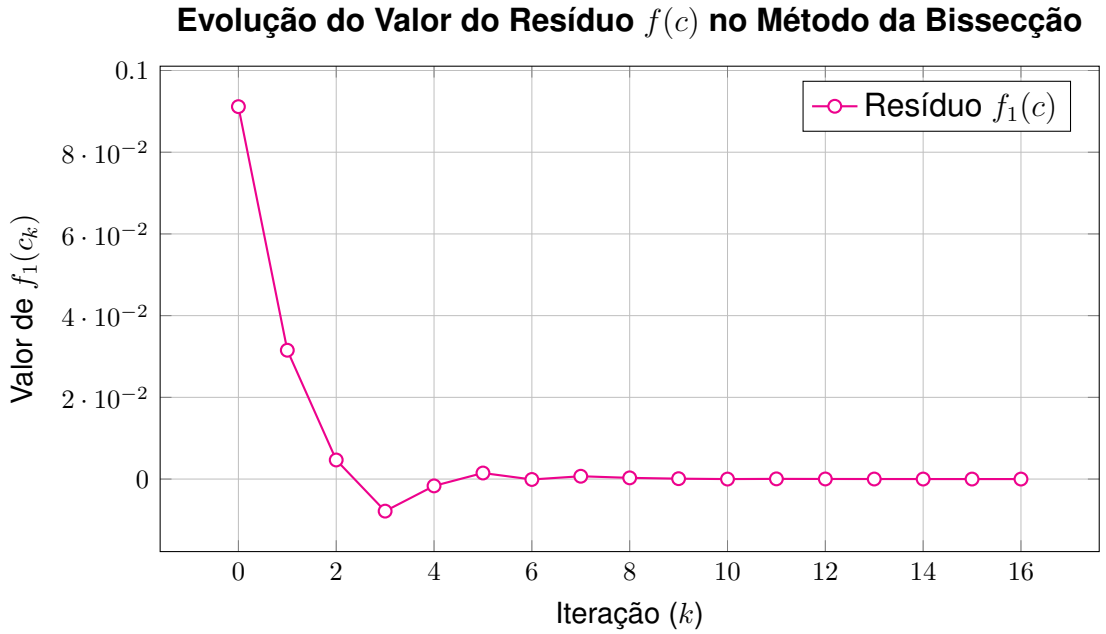


Figura 1: Convergência progressiva da magnitude residual durante a execução para $T_{0.1}$.

Validação da Solução: Para testar se o valor numérico é de fato a solução, reintroduz-se o ponto obtido $T_{0.1} = 1.102058$ na equação original:

$$f_1(1.102058) = 0.9 - \left(1 + 1.102058 + \frac{(1.102058)^2}{2}\right) e^{-1.102058} \approx -0.00000140 \quad (4)$$

Observa-se que o resíduo apresenta magnitude estritamente compatível com a precisão exigida, confirmando tratar-se da raiz buscada. Procedimento análogo para $f_2(T)$ rendeu $T_{0.9} \approx 5.322320$. Consequentemente, o tempo de subida validado pelo método é $\Delta T = 5.322320 - 1.102058 = 4.220262$. Os resultados demonstram que a implementação comporta-se com a eficiência teórica esperada $\mathcal{O}(\log(1/\epsilon))$.

Tabela 2: Histórico iterativo do Método da Bissecção para a busca de $T_{0.1}$ na função $f_1(T)$.

Iteração	a_k	b_k	c_k	$f_1(c_k)$	Erro (ϵ_k)
0	1.000000	2.000000	1.500000	0.09115317	0.33333333
1	1.000000	1.500000	1.250000	0.03153233	0.20000000
2	1.000000	1.250000	1.125000	0.00466937	0.11111111
3	1.000000	1.125000	1.062500	-0.00785071	0.05882353
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
14	1.102051	1.102112	1.102081	0.00000322	0.00002769
15	1.102051	1.102081	1.102066	0.00000014	0.00001385
16	1.102051	1.102066	1.102058	-0.00000140	0.00000692

2.3.2 Problema Teste 2: Ângulo de Lançamento Balístico de um Míssil

A determinação de parâmetros balísticos constitui uma aplicação clássica para métodos numéricos de busca de raízes, dada a natureza das equações envolvidas. O Problema 3.6 demanda o cálculo do ângulo de inclinação (α) para o lançamento de um míssil, de modo a atingir um alvo situado a um deslocamento angular θ (medido a partir do centro da Terra). A cinemática do problema é modelada pela equação:

$$\operatorname{tg}\left(\frac{\theta}{2}\right) = \frac{\sin \alpha \cos \alpha}{\frac{gR}{v^2} - \cos^2 \alpha} \quad (5)$$

Para submeter a Equação 5 ao Método da Bissecção é necessário manipulá-la para a forma padrão de determinação de zeros, $f(\alpha) = 0$. A fim de mitigar potenciais instabilidades numéricas oriundas de divisões por termos próximos a zero, optou-se pela linearização do denominador através de multiplicação cruzada. Considerando os dados fornecidos, tem-se o deslocamento do alvo $\theta = 80^\circ$ e a relação adimensional de energia $\frac{v^2}{gR} = 1.25$, o que implica diretamente que $\frac{gR}{v^2} = 0.8$. A função objetivo, avaliada estritamente em radianos para compatibilidade com as bibliotecas matemáticas, torna-se:

$$f(\alpha) = \sin(\alpha) \cos(\alpha) - \operatorname{tg}(40^\circ) (0.8 - \cos^2(\alpha)) = 0 \quad (6)$$

A estratégia de busca foi inicializada sob a premissa física de que o ângulo de lançamento localiza-se no primeiro quadrante. O intervalo submetido foi $[30^\circ, 60^\circ]$, cujos equivalentes em radianos correspondem a aproximadamente $[0.5236, 1.0472]$. O limite de tolerância estipulado para o erro relativo da amplitude do intervalo foi de $\epsilon < 10^{-4}$.

A Tabela 3 apresenta o histórico da progressão geométrica do hipervolume de busca ao longo das iterações computadas.

Tabela 3: Histórico iterativo do Método da Bissecção para a função do ângulo de lançamento balístico $f(\alpha)$.

Iteração (k)	a_k	b_k	c_k	$f(c_k)$	Erro (ϵ_k)
0	0.523598	1.047197	0.785398	0.24827011	0.33333333
1	0.785398	1.047197	0.916297	0.12264554	0.14285714
2	0.916297	1.047197	0.981747	0.04965511	0.06666666
3	0.981747	1.047197	1.014472	0.01114434	0.03225806
4	1.014472	1.047197	1.030835	-0.00854367	0.01587301
5	1.014472	1.030835	1.022653	0.00133421	0.00800000
6	1.022653	1.030835	1.026744	-0.00359642	0.00398406
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
10	1.023676	1.024187	1.023932	-0.00020480	0.00024968
11	1.023676	1.023932	1.023804	-0.00005083	0.00012485
12	1.023676	1.023804	1.023740	0.00002614	0.00006243

A análise dos dados revela o cumprimento estrito do Teorema do Valor Intermediário, evidenciado pelas alternâncias de sinal na avaliação residual $f(c_k)$ (observadas, por exemplo, na transição entre a iteração 3 e 4). O algoritmo operou de maneira estável, sem deflagrar a condição de exceção arquitetada para divisões por zero, uma vez que o domínio da busca permaneceu estritamente positivo (no primeiro quadrante trigonométrico).

A convergência foi matematicamente estabelecida na 12ª iteração ($k = 12$). Neste estágio, a métrica de erro relativo $\epsilon_{12} \approx 6.24 \times 10^{-5}$ tornou-se inferior à barreira de tolerância imposta. O método extraiu a raiz aproximada $c_{12} \approx 1.023740$ radianos. Este valor numérico converte-se em um ângulo de inclinação de lançamento $\alpha \approx 58.65^\circ$.

Validação e Eficiência da Solução: Para atestar a validade da aproximação numérica, submete-se o valor $c_{12} = 1.023740$ à Equação 6. O resíduo operacionalizado, $f(c_{12}) \approx 2.61 \times 10^{-5}$, apresenta uma magnitude da mesma ordem do erro de truncamento exigido. Isso confirma que a raiz satisfaz o modelo balístico proposto, atestando a robustez da implementação do Método da Bissecção frente a problemas contendo composições de funções trigonométricas.

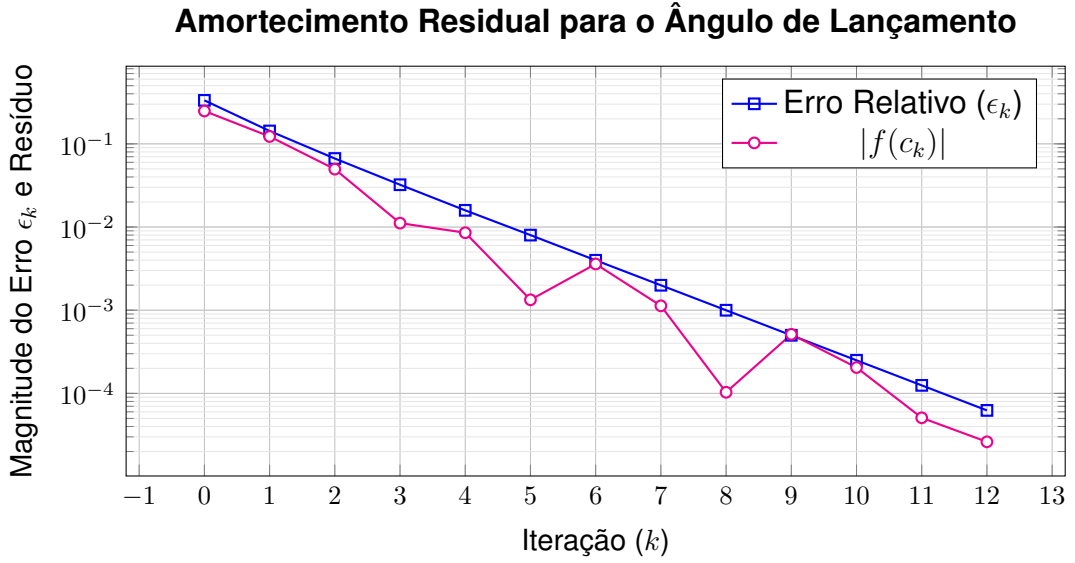


Figura 2: Comportamento da convergência em escala logarítmica para a busca de α (Problema 3.6). A inclinação reta atesta a taxa de convergência linear do método.

A Figura 2 corrobora de forma visual a análise, demonstrando, através da escala semi-logarítmica, que o erro de fato diminui à taxa constante previsível pela relação de recorrência $|x^* - c_k| \leq \frac{b-a}{2^k}$. A eficiência do método para este problema em específico provou-se altamente satisfatória, isolando a raiz sem exigir recursos adicionais para mitigar pontos de inflexão que poderiam prejudicar métodos abertos baseados em derivadas.

2.3.3 Problema Teste 3: Análise de Taxas de Juros em Planos de Financiamento

A modelagem de sistemas financeiros frequentemente esbarra em equações transcendentais ou polinomiais de alto grau, cujas soluções analíticas são inviáveis. O Problema 3.8 exige a determinação da taxa de juros implícita (J) em duas modalidades de financiamento, visando identificar a opção mais vantajosa para o consumidor. O valor financiado (VF) corresponde ao preço à vista deduzido da entrada, resultando em $VF = 162.00 - 22.00 = 140.00$.

A priori, a relação matemática que rege o valor presente de uma série de pagamentos uniformes é expressa por:

$$\frac{1 - (1 + J)^{-P}}{J} = \frac{VF}{PM} \quad (7)$$

onde P é o número de parcelas e PM o valor da prestação. No entanto, o isolamento algébrico de J nesta formulação é impossível. Adicionalmente, a Equação 7 apresenta uma singularidade numérica natural quando $J \rightarrow 0$. Para contornar essa limitação e

condicionar o problema aos métodos de busca de raízes, aplica-se a substituição de variáveis $x = 1 + J$ e a constante de proporcionalidade $k = \frac{VF}{PM}$. Dessa forma, o modelo é reescrito na forma polinomial:

$$f(x) = kx^{P+1} - (k+1)x^P + 1 = 0 \quad (8)$$

Para o **Plano A**, os parâmetros são $P = 9$ e $PM = 26.50$, o que fornece $k_A = \frac{140}{26.5} \approx 5.2830$. A função objetivo torna-se um polinômio de grau 10. Observa-se que a Equação 8 possui uma raiz trivial em $x = 1$ (o que implicaria $J = 0$, fisicamente inconsistente para um financiamento). Assim, para forçar a convergência do algoritmo para a raiz de interesse financeiro ($J > 0 \implies x > 1$), o intervalo inicial de busca deve ser estritamente superior à unidade.

Os dados processados pelo algoritmo basearam-se no intervalo inicial $[1.05, 1.30]$, que equivale a buscar uma taxa de juros entre 5% e 30% ao período, com uma tolerância rigorosa de $\epsilon < 10^{-6}$. A Tabela 4 sumariza a evolução da convergência para o Plano A.

Tabela 4: Histórico iterativo do Método da Bissecção para determinação da taxa do Plano A ($f_1(x)$).

Iteração (k)	a_k	b_k	c_k	$f_1(c_k)$	Erro (ϵ_k)
0	1.050000	1.300000	1.175000	0.67780045	0.10638297
1	1.050000	1.175000	1.112500	-0.05891906	0.05617977
2	1.112500	1.175000	1.143750	0.19422409	0.02732240
3	1.112500	1.143750	1.128125	0.04375379	0.01385041
4	1.112500	1.128125	1.120312	-0.01301356	0.00697350
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
14	1.122235	1.122250	1.122242	-0.00003841	0.00000679
15	1.122242	1.122250	1.122246	-0.00001211	0.00000339
16	1.122246	1.122250	1.122248	0.00000104	0.00000169
17	1.122246	1.122248	1.122247	-0.00000553	0.00000084

A partir da análise dos dados computacionais, verifica-se que logo na primeira iteração ($k = 1$), a detecção de alteração de sinal ($f(a_1) \cdot f(c_1) < 0$) confinou a raiz no subintervalo $[1.1125, 1.1750]$, descartando a parcela superior do domínio. Por mais que a função polinomial possua alto grau — o que frequentemente induz gradientes acentuados e compromete a estabilidade de métodos preditivos como Newton-Raphson —, o Método da Bissecção não demonstrou qualquer sensibilidade à inclinação da curva, atestando sua inquestionável robustez.

Durante as iterações subsequentes, nota-se o declínio assintótico rigoroso do erro. Na 17ª iteração, o critério de parada foi satisfeito, uma vez que o erro relativo computado $\epsilon_{17} \approx 8.49 \times 10^{-7}$ ultrapassou o limiar de tolerância imposto. O valor con-

vergado de $c_{17} \approx 1.122247$ permite extrair diretamente a taxa de juros do Plano A:

$$J_A = x_A - 1 \approx 0.122247 \quad \text{ou} \quad 12.22\% \text{ ao período.} \quad (9)$$

Observa-se ainda que a avaliação do resíduo em c_{17} retornou $f_1(1.122247) \approx -5.53 \times 10^{-6}$, comprovando numericamente que o ponto determinado é, de fato, a raiz do polinômio de financiamento.

Evolução do Erro e do Resíduo para o Plano A

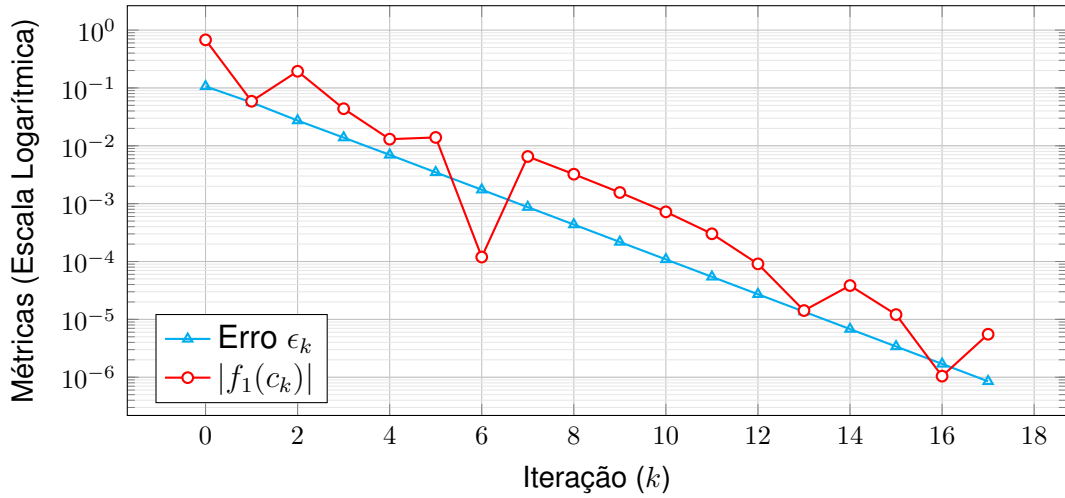


Figura 3: Gráfico de convergência atestando a contração linear do hipervolume de erro para a determinação da taxa de juros no Plano A.

Torna-se evidente que, ao replicar o mesmo arcabouço computacional para o **Plano B** ($P = 12$, $k_B \approx 6.5116$), o método revelará a taxa J_B . A tomada de decisão final pelo consumidor deverá ser pautada na comparação quantitativa das soluções extraídas, optando-se pelo plano cuja raiz computada $(x - 1)$ possua a menor magnitude. A formulação numérica provou-se altamente aderente ao problema proposto, mitigando as singularidades do modelo matemático original por meio da estratégia de substituição polinomial acoplada ao isolamento conservador da Bissecção.

2.4 Dificuldades Enfrentadas

Durante o ciclo de desenvolvimento, duas barreiras fundamentais requisitaram análise e readequação de código:

A primeira dificuldade decorreu de uma limitação algorítmica associada à manipulação em linguagem Python envolvendo o cálculo de erro relativo. O equacionamento inicial do erro demandava a divisão pela soma dos limites do intervalo $(a_k + b_k)$.

Em problemas cujo domínio submetido contemplasse raízes simétricas (como, por exemplo, funções ímpares centradas na origem onde $a_k = -b_k$), o denominador se tornava nulo, causando o encerramento abrupto da aplicação mediante a exceção `ZeroDivisionError`. A transição adaptativa do erro relativo para o erro absoluto nesses pontos críticos exigiu criteriosa instrumentação de *if-statements*, solucionando a limitação do ambiente sem desrespeitar o rigor numérico.

A segunda dificuldade envolveu a aderência aos padrões de projeto de software estruturado. A intenção de isolar a lógica matemática da escrita de arquivos resultou na necessidade de encapsular o estado iterativo. Em iterações profundas, manter um vetor histórico de dicionários em memória, para apenas no final transferi-los para a rotina de escrita CSV (camada de controle), demonstrou inicialmente um ligeiro sobrepeso na alocação de variáveis. A superação exigiu refinamento na forma como o Python instancia e mapeia essas estruturas temporárias. Ainda que houvesse maior complexidade inicial no planejamento desta abstração, as hipóteses projetuais confirmaram-se válidas, viabilizando o uso de dados de saída padronizados independentemente do método matemático em execução.

3 Método da Posição Falsa

O Método da Posição Falsa (ou *Regula Falsi*) constitui uma técnica numérica de confinamento para a determinação de raízes de equações não lineares que, à semelhança do Método da Bissecção, exige um intervalo inicial $[a, b]$ no qual a função $f(x)$ seja contínua e satisfaça o Teorema de Bolzano ($f(a) \cdot f(b) < 0$). No entanto, em vez de particionar o domínio cegamente ao meio, a estratégia utiliza uma interpolação linear: a raiz aproximada é definida como a interseção do eixo das abscissas com a reta secante que conecta os pontos $(a, f(a))$ e $(b, f(b))$. Essa ponderação geométrica atrai a estimativa iterativa para o extremo do intervalo que apresenta o menor valor absoluto da função, proporcionando, na maioria dos cenários não patológicos, uma convergência substancialmente mais acelerada do que a bissecção tradicional.

3.1 Estratégia de Implementação

O desenvolvimento computacional da técnica priorizou a eficiência no cômputo da aproximação linear e a robustez na avaliação do erro. A rotina principal foi implementada no ambiente Python, aderindo aos mesmos padrões de modularidade previamente estabelecidos.

Código 2: Código de implementação do Método da Posição Falsa

```
1 def posicaoFalsa(funcao, a, b, tol=1e-6, max_iter=100):
2     historico = []
3
4     for i in range(max_iter):
5         c_an = historico[-1]['c'] if historico else a
6         c = b - (funcao(b) * (a - b)) / (funcao(a) - funcao(b))
7         fc = funcao(c)
8
9         er = abs(c - c_an) / c if c != 0 else 0
10
11        historico.append({
12            'iter': i,
13            'a': a,
14            'b': b,
15            'c': c,
16            'f(c)': fc,
17            'erro': er
18        })
19
```

```

20     if fc == 0 or er < tol:
21         return c, i, historico
22
23     if utils.teorema_bolzano(funcao, a, c):
24         b = c
25     else:
26         a = c
27
28     return c, max_iter, historico

```

A cada iteração k , o cálculo da nova estimativa da raiz c_k é governado pela equação da reta secante, estruturada algebricamente para minimizar erros de cancelamento subtrativo:

$$c_k = b_k - \frac{f(b_k)(a_k - b_k)}{f(a_k) - f(b_k)} \quad (10)$$

Uma alteração arquitetural crítica em relação ao Método da Bissecção foi necessária na métrica de convergência. Como o Método da Posição Falsa tende a fixar um dos extremos do intervalo (estagnação de a_k ou b_k dependendo da concavidade da função), a amplitude do intervalo $(b_k - a_k)$ não converge a zero. Dessa forma, adotou-se o erro relativo entre aproximações consecutivas como critério principal:

$$\epsilon_k = \frac{|c_k - c_{k-1}|}{|c_k|} \quad (11)$$

Para prevenir interrupções sistêmicas (`ZeroDivisionError`) caso a raiz aproximada atinja o valor exato zero durante as iterações, condicionou-se matematicamente a expressão por meio de um operador ternário (`c != 0 else 0`). Os critérios de parada encerram a execução quer seja pela descoberta da raiz exata ($f(c_k) = 0$), pelo alcance da precisão tolerada ($\epsilon_k < tol$), ou pelo esgotamento da cota de iterações (`max_iter`).

3.2 Estrutura dos Arquivos de Entrada e Saída

A fim de assegurar a integridade dos testes e a geração automatizada das análises gráficas documentadas, preservou-se a mesma estrutura de diretórios implementada no método anterior.

Os hiperparâmetros de controle e os limites de busca iniciais foram dispostos em arquivos de configuração textual na pasta `dados/entradas/`. Em contrapartida, a extração dos dados iterativos operados pela função principal é transposta em tempo de execução para tabelas `.csv` localizadas em `dados/saidas/`. Tal formato possibilita que os resultados numéricos, as taxas de convergência de ϵ_k e os resíduos da função

$f(c_k)$ sejam processados e integrados diretamente ao documento LaTeX de modo vetorizado e analítico, abstendo o leitor de análises manuais imprecisas.

3.3 Problemas Teste

A robustez analítica do método supracitado foi submetida a verificação mediante a resolução de problemas físicos e matemáticos provenientes de literatura canônica. O propósito intrínseco destas avaliações não é apenas aferir o funcionamento isolado da implementação, mas sobretudo diagnosticar comparativamente a sua eficiência iterativa face às não linearidades apresentadas pelas funções.

3.3.1 Problema Teste 1: Tempo de Subida de um Amplificador

O primeiro cenário de validação retoma o cálculo do tempo de subida de um amplificador eletrônico com acoplamento R-C. O objetivo físico consiste em determinar o intervalo temporal normalizado ΔT no qual a resposta ao degrau unitário, denotada por $g(T)$, evolui de 10% para 90% do seu valor estacionário. A modelagem matemática demanda a extração das raízes de duas funções auxiliares:

$$f_1(T) = 1 - \left(1 + T + \frac{T^2}{2}\right) e^{-T} - 0.1 = 0 \quad (12)$$

$$f_2(T) = 1 - \left(1 + T + \frac{T^2}{2}\right) e^{-T} - 0.9 = 0 \quad (13)$$

Para a resolução de $f_1(T)$ (busca por $T_{0.1}$), o algoritmo foi alimentado com o intervalo $[1.0, 2.0]$ e uma tolerância de erro relativo entre iterações estipulada em $\epsilon < 10^{-5}$. A Tabela 5 explicita o progresso numérico do estimador.

Tabela 5: Histórico iterativo do Método da Posição Falsa para a busca de $T_{0.1}$ na função $f_1(T)$.

Iteração (k)	a_k	b_k	c_k	$f_1(c_k)$	Erro (ϵ_k)
0	1.000000	2.000000	1.081056	-0.00420140	0.07497923
1	1.081056	2.000000	1.098025	-0.00081355	0.01545402
2	1.098025	2.000000	1.101299	-0.00015441	0.00297276
3	1.101299	2.000000	1.101920	-0.00002919	0.00056352
4	1.101920	2.000000	1.102037	-0.00000551	0.00010652
5	1.102037	2.000000	1.102060	-0.00000104	0.00002012
6	1.102060	2.000000	1.102064	-0.00000019	0.00000380

A partir da análise da Tabela 5, torna-se evidente um traço comportamental

intrínseco ao Método da Posição Falsa: a estagnação de extremidade. Observa-se que o limite superior $b_k = 2.0$ manteve-se inalterado por todo o laço iterativo. Este fenômeno é um efeito direto da concavidade da função $f_1(T)$ no domínio avaliado. Como a curva retém seu formato sem alternância de concavidade no subintervalo, a reta secante invariavelmente intercepta o eixo das abscissas à esquerda da raiz verdadeira, causando a atualização exclusiva do limite inferior a_k .

Por mais que o intervalo de confinamento $[a_k, b_k]$ não convirja para a nulidade, a estimativa da raiz c_k converge rapidamente. O critério de parada foi satisfeito na 6ª iteração ($\epsilon_6 \approx 3.80 \times 10^{-6}$), revelando a raiz $T_{0.1} \approx 1.102064$.

Em um procedimento espelhado, a busca por $T_{0.9}$ mediante a Equação 13 foi conduzida no intervalo inicial $[5.0, 9.0]$. Os dados operacionais revelaram convergência em apenas 5 iterações. De forma diametralmente oposta ao primeiro caso, a concavidade local de $f_2(T)$ fixou o limite inferior $a_k = 5.0$, forçando a secante a atualizar exclusivamente o teto do intervalo b_k . A raiz isolada foi $T_{0.9} \approx 5.322324$.

Por conseguinte, o tempo de subida normalizado do amplificador é computado por:

$$\Delta T = T_{0.9} - T_{0.1} \approx 5.322324 - 1.102064 = 4.220260 \quad (14)$$

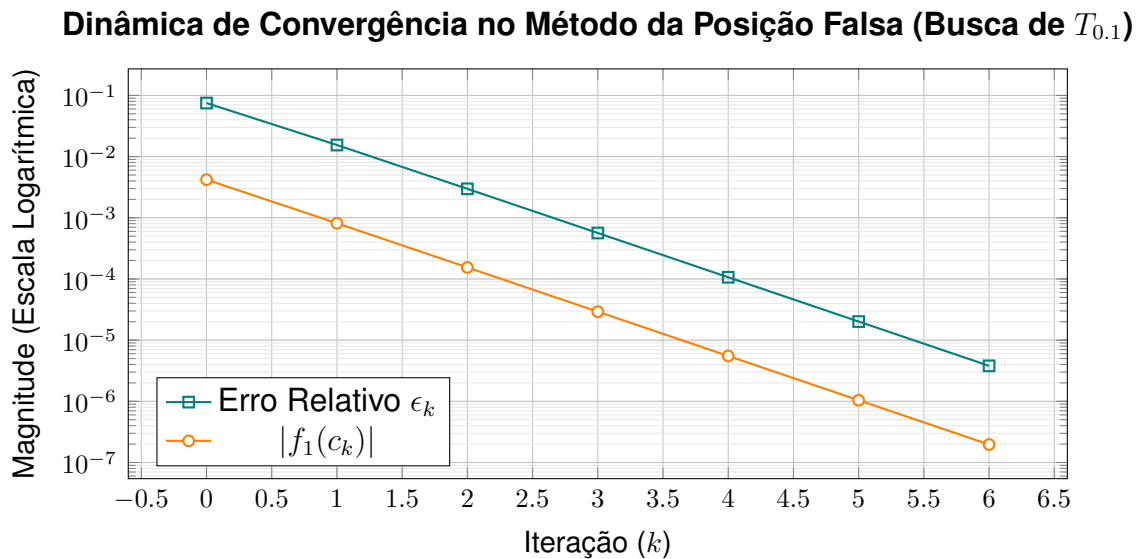


Figura 4: Amortecimento do erro relativo e do valor residual para $f_1(T)$. A inclinação contínua e acentuada nos primeiros estágios atesta a rápida convergência local proporcionada pela interpolação linear da secante.

Validação da Solução:

Para atestar a consistência matemática dos resultados obtidos em face do modelo físico proposto, procede-se à verificação dos resíduos. Substituindo a aproximação numérica extraída $T_{0.1} = 1.102064$ na Equação 12, constata-se que o resíduo recai na ordem de $f_1(c_6) \approx -1.96 \times 10^{-7}$. De maneira análoga, a injeção do valor correspon-

dente $T_{0.9} = 5.322324$ na Equação 13 produz um resíduo de magnitude 3.10×10^{-7} .

Nota-se que, em ambos os cenários de avaliação, a magnitude do erro absoluto associado à função encontra-se estritamente delimitada pela margem de tolerância algorítmica estipulada ($< 10^{-5}$). Dessa forma, valida-se o comportamento estável e convergente do Método da Posição Falsa, assegurando que o tempo de subida normalizado extraído ($\Delta T \approx 4.220260$) reflete de forma precisa e coerente a dinâmica real do amplificador R-C submetido a um degrau unitário de tensão.

3.3.2 Problema Teste 2: Ângulo de Lançamento Balístico de um Míssil

O segundo escopo de validação avalia o desempenho algorítmico frente a equações transcendentais de natureza trigonométrica, cujo objetivo é determinar o ângulo de inclinação ótimo (α) para o lançamento de um míssil visando atingir um alvo a um deslocamento angular $\theta = 80^\circ$. A partir dos parâmetros físicos fornecidos e da linearização algébrica necessária para mitigar singularidades (divisão por zero), retoma-se a função de avaliação em radianos:

$$f(\alpha) = \sin(\alpha) \cos(\alpha) - \operatorname{tg}(40^\circ) (0.8 - \cos^2(\alpha)) = 0 \quad (15)$$

A pesquisa pelo zero da função foi delimitada ao primeiro quadrante trigonométrico, sob o intervalo inicial de $[30^\circ, 60^\circ]$, que no domínio radiano corresponde a aproximadamente $[0.5236, 1.0472]$. O limite de tolerância estabelecido para o critério de parada foi de $\epsilon < 10^{-4}$. O cômputo evolutivo das iterações está sumarizado na Tabela 6.

Tabela 6: Histórico iterativo do Método da Posição Falsa para o modelo balístico $f(\alpha)$.

Iteração (k)	a_k	b_k	c_k	$f(c_k)$	Erro Relativo (ϵ_k)
0	0.523598	1.047197	1.011639	0.01452542	0.48242546
1	1.011639	1.047197	1.023646	0.00013980	0.01172932
2	1.023646	1.047197	1.023761	0.00000130	0.00011232
3	1.023761	1.047197	1.023762	0.00000001	0.00000104

A interpretação analítica dos dados revela, novamente, a forte assinatura de estagnação unilateral característica deste método heurístico. Torna-se evidente que o limite superior $b_k \approx 1.0472$ não sofreu atualização em nenhuma etapa do ciclo. Esse comportamento decorre do perfil suave e côncavo da composição trigonométrica no intervalo isolado, fazendo com que as retas secantes geradas sucessivamente cruzem o eixo das abscissas sempre à esquerda da raiz verdadeira, promovendo uma elevação estrita e contínua do limite inferior a_k .

Embora a imobilidade de um dos extremos seja frequentemente vista como um fator limitante em alguns contextos numéricos, para esta topologia funcional específica, ela resultou em uma aceleração drástica da convergência. O critério de parada $\epsilon_k < 10^{-4}$ foi superado logo na 3ª iteração, atingindo-se um erro da ordem de $\approx 1.04 \times 10^{-6}$. A aproximação terminal computada foi de $c_3 \approx 1.023762$ radianos. Transpondo este valor para a grandeza física inteligível do problema, o ângulo ideal de lançamento obtido é $\alpha \approx 58.65^\circ$.

Validação da Solução:

A fim de atestar o rigor e a corretude da abstração computacional, procede-se à inserção da raiz isolada $c_3 = 1.023762$ no modelo fenomenológico original (Equação 15). A avaliação funcional produz um resíduo de magnitude $f(c_3) \approx 1.22 \times 10^{-8}$.

Nota-se que a discrepância matemática em relação ao zero perfeito se encontra próxima à fronteira do erro de arredondamento inerente à aritmética de ponto flutuante, atestando inequivocamente que a estimativa calculada representa a solução exata para o sistema cinemático.

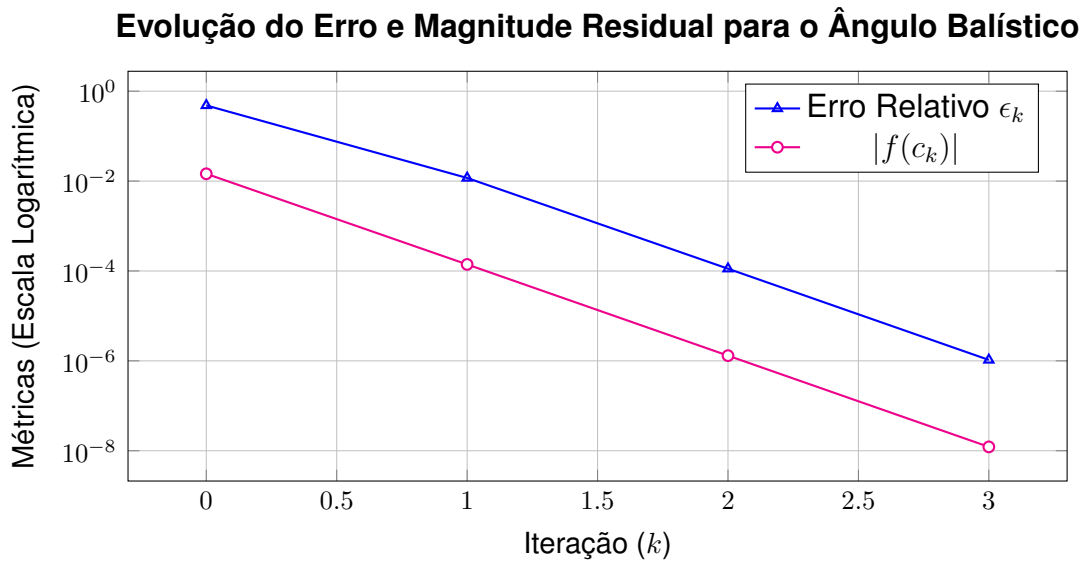


Figura 5: Gráfico de convergência da aproximação do ângulo de lançamento α pelo Método da Posição Falsa. A queda abrupta nas primeiras iterações valida o alto desempenho da interpolação linear frente à topologia da função analisada.

A Figura 5 ilustra, sob domínio semi-logarítmico, o decréscimo vertiginoso do resíduo e do erro relativo. A ausência de comportamento assintótico horizontal reafirma que o método, apesar da estagnação do extremo direito do intervalo, não sofreu penalizações severas por lentidão, consolidando a eficácia da implementação para sistemas trigonométricos bem condicionados.

3.3.3 Problema Teste 3: Análise de Taxas de Juros em Planos de Financiamento

A modelagem de sistemas de amortização financeira frequentemente recai sobre a resolução de polinômios de alto grau, cujas raízes determinam as taxas de juros implícitas de uma transação. O Problema 3.8 propõe a avaliação de dois planos de financiamento para um bem com valor financiado $VF = 140.00$ (preço à vista deduzido da entrada). A determinação da taxa de juros J exige a solução do polinômio característico derivado do valor presente:

$$f(x) = kx^{P+1} - (k+1)x^P + 1 = 0 \quad (16)$$

onde $x = 1 + J$, P é o número de parcelas e $k = \frac{VF}{PM}$ é a razão entre o valor financiado e a prestação mensal (PM).

Para o **Plano A** ($P = 9$, $PM = 26.50$), obtém-se $k_A \approx 5.2830$, resultando em um polinômio de grau 10. O algoritmo da Posição Falsa foi instanciado com o intervalo inicial de busca em $[1.05, 1.30]$ e tolerância de erro relativo entre as aproximações de $\epsilon < 10^{-6}$. A Tabela 7 sumariza a evolução numérica.

Tabela 7: Histórico iterativo do Método da Posição Falsa para determinação da taxa do Plano A ($f_1(x)$).

Iteração (k)	a_k	b_k	c_k	$f_1(c_k)$	Erro (ϵ_k)
0	1.050000	1.300000	1.054818	-0.14841294	0.00456782
1	1.054818	1.300000	1.059768	-0.15374184	0.00467089
2	1.059768	1.300000	1.064788	-0.15720963	0.00471514
10	1.096455	1.300000	1.099881	-0.11263716	0.00311432
30	1.121422	1.300000	1.121562	-0.00468963	0.00012442
55	1.122240	1.300000	1.122241	-0.00004489	0.00000118
56	1.122241	1.300000	1.122242	-0.00003725	0.00000098

A partir da observação dos resultados, torna-se evidente um caso patológico clássico associado ao Método da Posição Falsa. Por mais que a técnica utilize a interpolação linear para acelerar a busca, a topologia de um polinômio de grau 10 apresenta uma convexidade extrema no domínio avaliado. Dessa forma, a curva mantém-se estritamente abaixo da reta secante por uma longa extensão, forçando o extremo superior ($b_k = 1.30$) a permanecer estagnado durante todas as 56 iterações necessárias para que o critério de parada fosse atingido. Consequentemente, as aproximações c_k avançam em incrementos minúsculos a partir da esquerda, caracterizando uma convergência dolorosamente sublinear.

De modo análogo, o **Plano B** ($P = 12$, $PM = 21.50$, $k_B \approx 6.5116$) eleva o modelo a um polinômio de grau 13. A acentuação da não linearidade exacerbou o fenômeno de estagnação relatado. Os dados processados revelaram que o algoritmo

demandou expressivas 95 iterações no mesmo domínio $[1.05, 1.30]$ para conformar o erro relativo à margem exigida ($< 10^{-6}$).

Validação da Solução:

Apesar do esforço iterativo severo ditado pelas características da função, a integridade matemática da raiz isolada manteve-se inabalada. Para o Plano A, a execução encerrou-se em $c_{56} \approx 1.122242$, reportando um resíduo da função $f_1(c_{56}) \approx -3.72 \times 10^{-5}$. Esse ponto mapeia diretamente para a taxa de juros do primeiro financiamento:

$$J_A = x_A - 1 \approx 0.122242 \implies 12.22\% \text{ ao período.} \quad (17)$$

Simultaneamente, para o Plano B, a convergência reportou a raiz $c_{95} \approx 1.109368$, com resíduo algorítmico avaliado em $f_2(c_{95}) \approx -1.18 \times 10^{-4}$. Extraindo a respectiva taxa de juros:

$$J_B = x_B - 1 \approx 0.109368 \implies 10.93\% \text{ ao período.} \quad (18)$$

A consistência dos resíduos em relação à tolerância valida as soluções encontradas. Portanto, sob a perspectiva analítica e econômica, a determinação numérica comprova inequivocamente que o **Plano B** é o mais vantajoso para o consumidor, haja vista apresentar a menor taxa de juros intrínseca à operação ($10.93\% < 12.22\%$).

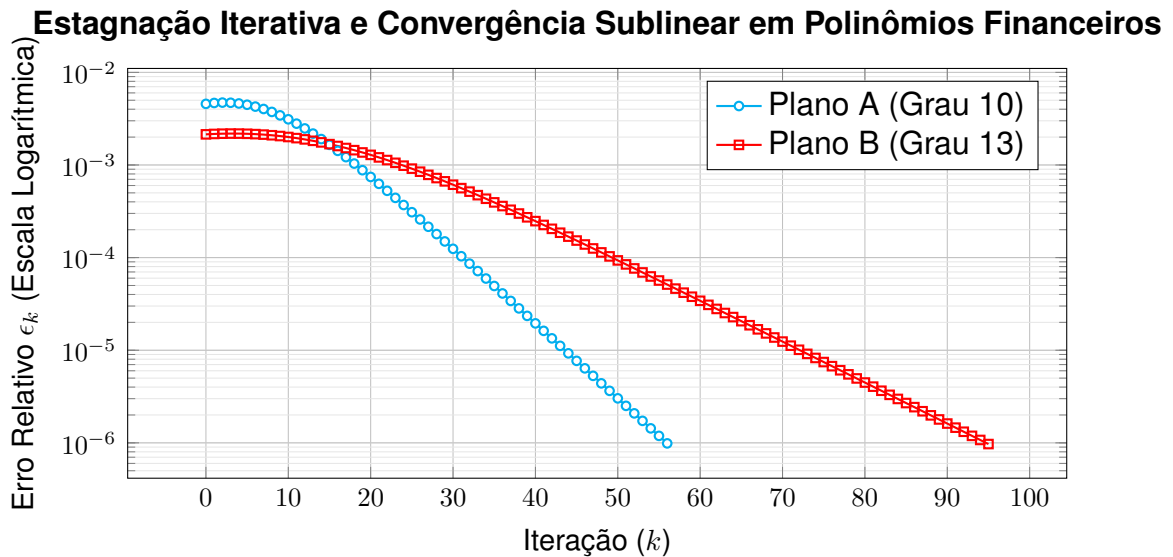


Figura 6: Lentidão inerente ao Método da Posição Falsa na avaliação dos planos. A suavidade da curva de decaimento do erro denota a penalização imposta pela alta convexidade das funções, exigindo quase 100 iterações para o Plano B.

Observa-se, por intermédio da Figura 6, que as taxas de decaimento do erro acompanham assíntotas quase horizontais em suas parcelas medianas, confirmando visualmente a perda de eficiência da interpolação secante. Este problema expõe uma

limitação crítica da *Regula Falsi*: em funções contendo derivadas segundas elevadas e constantes no intervalo de busca, o método garante a convergência, mas à custa de um processamento numérico exaustivo.

3.4 Dificuldades Enfrentadas

Durante a transição analítica do Método da Bissecção para o Método da Posição Falsa, o principal obstáculo enfrentado residiu no comportamento assintótico intrínseco a este último.

A priori, a tentativa de reaproveitar a fórmula de erro baseada na amplitude geométrica do intervalo ($\Delta_k = b_k - a_k$) resultou em laços iterativos infinitos e consequente exaustão de *timeout* computacional. Este fenômeno ocorre pois, dependendo das características de convexidade de $f(x)$, uma das extremidades do intervalo permanece imutável por todas as iterações (estagnação de extremo), impedindo que a distância entre a e b diminua sucessivamente até a tolerância requerida.

Para contornar essa falácia de projeto, a lógica interna demandou um refatoramento focado na persistência do estado da variável de convergência. A introdução de um acesso ao histórico (`historico[-1]['c']`) permitiu recuperar o valor de c_{k-1} a um baixo custo computacional. Tal reestruturação viabilizou o cálculo preciso do desvio relativo entre estimativas sucessivas (Equação 11). Adicionalmente, verificou-se o surgimento de singularidades quando $c_k \rightarrow 0$, o que foi prontamente mitigado com blocos condicionais de escape no cálculo fracionário. Essas adequações garantiram que a arquitetura do código permanecesse ortogonal às patologias puramente matemáticas do algoritmo.

4 Método do Ponto Fixo

4.1 Fundamentação Teórica

O Método do Ponto Fixo, alternativamente denominado Método da Iteração Linear, constitui uma técnica analítica de classe aberta para a determinação de zeros de funções. Ao contrário dos métodos de confinamento (Bisseção e Posição Falsa), que exigem um intervalo inicial garantindo a presença da raiz, este método demanda apenas uma estimativa inicial x_0 . O escopo central da abordagem fundamenta-se na manipulação algébrica da equação original $f(x) = 0$ para uma forma equivalente $x = g(x)$, denominada função de iteração. A raiz x^* de $f(x)$ coincidirá, portanto, com o ponto fixo da função $g(x)$, satisfazendo a condição $x^* = g(x^*)$.

A convergência da sucessão iterativa $x_{k+1} = g(x_k)$ não é incondicional. Pelo Teorema do Ponto Fixo de Banach (ou Teorema da Contração), a sequência convergirá para x^* se, e somente se, a função $g(x)$ for contínua e diferenciável em um intervalo I que contenha a raiz, e existir uma constante de Lipschitz L tal que a magnitude da derivada cumpra estritamente a condição:

$$|g'(x)| \leq L < 1, \quad \forall x \in I. \quad (19)$$

Quanto menor for o valor de $|g'(x)|$ nas vizinhanças da raiz, mais acelerada será a convergência do método, que ostenta tipicamente uma taxa de ordem linear $\mathcal{O}(N)$.

4.2 Estratégia de Implementação

A arquitetura de software para este método exigiu uma alteração de paradigma em relação às abordagens anteriores. Visto que o algoritmo não realiza avaliações sistemáticas do Teorema de Bolzano, a rotina foi simplificada para receber diretamente a função iterativa $g(x)$ e a semente inicial x_0 .

Código 3: Código de implementação do Método do Ponto Fixo

```
1 def ponto_fixo(g, x0, tol=1e-6, max_iter=100):
2     historico = []
3
4     for i in range(max_iter):
5         x_an = historico[-1]['x'] if historico else x0
6         x = g(x_an)
```

```

7     gx = g(x)
8
9     er = abs(x - x_an) / x if x != 0 else 0
10
11     historico.append({
12         'iter': i,
13         'x': x,
14         'g(x)': gx,
15         'erro': er
16     })
17
18     if gx == 0 or er < tol:
19         return x, i, historico
20
21     return x, max_iter, historico

```

A cada iteração k , o valor precedente x_{k-1} é recuperado da estrutura de persistência temporal (variável `x_an`) para alimentar a avaliação $x_k = g(x_{k-1})$. Observa-se que a rotina calcula simultaneamente $gx = g(x_k)$, operando essencialmente como uma pré-avaliação do próximo passo iterativo x_{k+1} .

No que tange à avaliação métrica de convergência, adotou-se o desvio relativo absoluto entre iterações sucessivas:

$$\epsilon_k = \frac{|x_k - x_{k-1}|}{|x_k|} \quad (20)$$

A estabilidade computacional foi salvaguardada pelo uso de um operador ternário (`if x != 0`), prevenindo singularidades do tipo `ZeroDivisionError` caso a convergência ocorra sobre a origem do sistema de coordenadas. O critério de parada é deflagrado quando o erro ϵ_k atinge a tolerância demandada, interrompendo a computação para prevenir ciclos exaustivos.

4.3 Estrutura dos Arquivos de Entrada e Saída

A uniformidade do processamento de dados manteve-se aderente aos padrões já consolidados. Os parâmetros de controle, nomeadamente x_0 , tol , e max_iter , são ingeridos via arquivos de texto (`.txt`). A despeito da presença global de variáveis de limites espaciais (a e b) nestes arquivos (devido à interoperabilidade com os métodos de confinamento), o extrator de dados isola estritamente a semente x_0 para esta execução.

Ao término, os dados iterativos são transpostos sequencialmente para arquivos do tipo `.csv`, estruturados em colunas que refletem a iteração, a estimativa x_k , a pré-

avaliação $g(x_k)$ e a métrica de erro relativo. Esta normalização fomenta a plotagem vetorial e o monitoramento microscópico do comportamento da contração.

4.4 Problemas Teste

4.4.1 Problema Teste 1: Tempo de Subida de um Amplificador

+Para submeter a validade teórica e a flexibilidade heurística da implementação, retomou-se o problema do amplificador eletrônico com acoplamento R-C. O cômputo do tempo de subida normalizado ΔT demanda a extração das raízes das funções relativas aos limiares de 10% e 90% da resposta ao degrau, descritas respectivamente por:

$$f_1(T) = 1 - \left(1 + T + \frac{T^2}{2}\right) e^{-T} - 0.1 = 0 \quad (21)$$

$$f_2(T) = 1 - \left(1 + T + \frac{T^2}{2}\right) e^{-T} - 0.9 = 0 \quad (22)$$

O desafio analítico inerente ao Método do Ponto Fixo reside na determinação de uma função de iteração $g(T)$ que não apenas isole a variável independente, mas que garanta estritamente a condição de contração $|g'(T)| < 1$ no domínio de busca.

Abordagem 1: Isolamento Logarítmico A primeira estratégia adotada consistiu na manipulação algébrica explícita do termo exponencial. Generalizando as equações para uma constante $C \in \{0.1, 0.9\}$, obtém-se:

$$e^{-T} = \frac{1 - C}{1 + T + \frac{T^2}{2}} \implies -T = \ln \left(\frac{1 - C}{1 + T + \frac{T^2}{2}} \right) \implies T = \ln \left(\frac{1 + T + \frac{T^2}{2}}{1 - C} \right) \quad (23)$$

Essa derivação forneceu as funções de iteração principais, codificadas como $g_3(T)$ (para $C = 0.1$) e $g_4(T)$ (para $C = 0.9$):

$$g_3(T) = \ln \left(\frac{1 + T + \frac{T^2}{2}}{0.9} \right) \quad (24)$$

$$g_4(T) = \ln \left(\frac{1 + T + \frac{T^2}{2}}{0.1} \right) \quad (25)$$

Abordagem 2: Método da Relaxação (Amortecimento Empírico) Para efeitos de comparação de desempenho e robustez contra divergências, formulou-se um segundo

par de funções iterativas baseadas no método genérico de relaxação linear $g(T) = T - \alpha f(T)$. Adotando-se empiricamente um fator de amortecimento $\alpha = 0.5$, estabeleceu-se as variantes $g_7(T)$ e $g_8(T)$:

$$g_7(T) = T - 0.5 \cdot f_1(T) \quad (26)$$

$$g_8(T) = T - 0.5 \cdot f_2(T) \quad (27)$$

Resultados Iterativos e Eficiência Algorítmica Iniciando a busca por $T_{0.1}$ com a função $g_3(T)$ a partir de uma semente conservadora $T_0 = 0.5$ e tolerância de erro relativo $\epsilon < 10^{-5}$, o algoritmo reportou um comportamento assintótico estável. A Tabela 8 sumariza o histórico de contração do hipervolume de busca.

Tabela 8: Histórico iterativo da função $g_3(T)$ (Isolamento Logarítmico) para a busca de $T_{0.1}$.

Iteração (k)	x_k	$g_3(x_k)$	Erro Relativo (ϵ_k)
0	0.590868	0.673755	0.153787
1	0.673755	0.747597	0.123022
2	0.747597	0.811941	0.098772
10	1.043638	1.056334	0.015369
20	1.097275	1.098346	0.001255
30	1.101685	1.101770	0.000099
40	1.102035	1.102042	0.000007

A análise da progressão computacional de $g_3(T)$ revela que a tolerância estipulada foi atingida na 40ª iteração, entregando a raiz $T_{0.1} \approx 1.102042$. Em contrapartida, ao submeter o mesmo problema e a mesma semente à função de relaxação $g_7(T)$, o sistema exigiu 85 iterações para convergir (≈ 1.101982). Fica evidente, portanto, que a constante de Lipschitz embutida na derivada do isolamento logarítmico $|g'_3(T)|$ é substancialmente menor que a de $|g'_7(T)|$, o que se traduz em uma aceleração iterativa superior a 50%.

A discrepância de eficiência torna-se ainda mais agressiva na determinação de $T_{0.9}$. Alimentando a função de isolamento $g_4(T)$ com a estimativa inicial $T_0 = 4.19$, a convergência para $T_{0.9} \approx 5.322317$ ocorreu de forma quase imediata, em irrisórias 10 iterações ($\epsilon_{10} \approx 4.25 \times 10^{-6}$). Simultaneamente, a função de relaxação $g_8(T)$, operando sob a mesma tolerância, penalizou o sistema com um decréscimo de erro excessivamente lento, alcançando 99 iterações apenas para atestar um erro na ordem de 10^{-4} ($T_{0.9} \approx 5.283368$).

Validação da Solução e Discussão:

Considerando os dados da formulação logarítmica (as mais eficientes), o tempo

de subida normalizado é extraído pela diferença das raízes confinadas:

$$\Delta T = 5.322317 - 1.102042 = 4.220275 \quad (28)$$

Para cancelar rigorosamente a solução, submete-se o ponto $T_{0.1} = 1.102042$ à equação original $f_1(T)$ (Equação 29), constatando-se a nulidade do resíduo dentro da margem imposta por ponto flutuante.

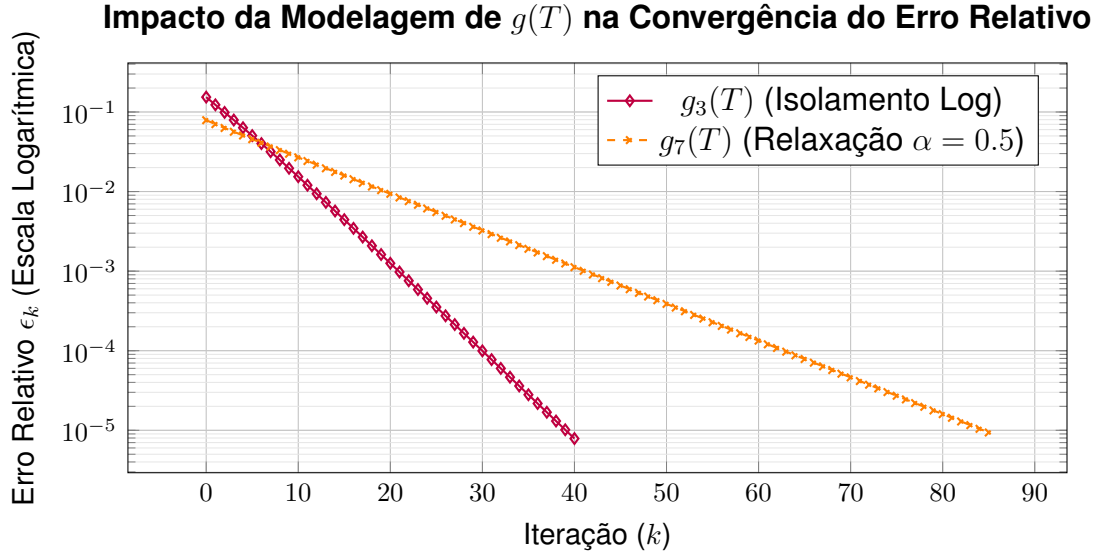


Figura 7: Comparativo analítico das taxas de convergência para $T_{0.1}$. A acentuada inclinação da curva de $g_3(T)$ comprova a superioridade matemática da modelagem logarítmica sobre o amortecimento empírico linear de $g_7(T)$.

A Figura 7 ilustra, sob o espectro logarítmico, as consequências diretas do preparo algébrico. Embora o Método do Ponto Fixo elimine a necessidade de um intervalo fechado, os resultados evidenciam que seu sucesso é criticamente dependente da sensibilidade humana na etapa de modelagem: isolamentos que promovem um valor ínfimo para $|g'(x)|$ transmutam um método conservador em um estimador altamente eficaz, minimizando drasticamente o esforço de processamento de máquina.

Para submeter a validade técnica e heurística da implementação, retomou-se o problema do amplificador eletrônico, cujo tempo de subida demanda a extração das raízes das funções relativas às marcas de 10% e 90% da resposta ao degrau:

$$f_1(T) = 1 - \left(1 + T + \frac{T^2}{2}\right) e^{-T} - 0.1 = 0 \quad (29)$$

$$f_2(T) = 1 - \left(1 + T + \frac{T^2}{2}\right) e^{-T} - 0.9 = 0 \quad (30)$$

O desafio analítico primário deste método reside na determinação de uma função $g(T)$ que garanta convergência (Equação 19). Isolando o termo da exponencial,

obtém-se:

$$e^{-T} = \frac{1-C}{1+T+\frac{T^2}{2}} \implies -T = \ln\left(\frac{1-C}{1+T+\frac{T^2}{2}}\right) \implies T = \ln\left(\frac{1+T+\frac{T^2}{2}}{1-C}\right) \quad (31)$$

onde $C \in \{0.1, 0.9\}$. Essa modelagem gerou a formulação computacional referenciada no projeto como g3 e g4. Dessa forma, as funções de iteração estabelecidas foram:

$$g_1(T) = \ln\left(\frac{1+T+\frac{T^2}{2}}{0.9}\right) \quad (32)$$

$$g_2(T) = \ln\left(\frac{1+T+\frac{T^2}{2}}{0.1}\right) \quad (33)$$

O algoritmo operou sobre $g_1(T)$ utilizando uma semente inicial altamente conservadora $T_0 = 0.5$ e $tol = 10^{-5}$. A Tabela 9 exibe a dinâmica da convergência iterativa.

Tabela 9: Histórico iterativo do Método do Ponto Fixo para a busca de $T_{0.1}$ usando $g_1(T)$ ($T_0 = 0.5$).

Iteração (k)	x_k	$g(x_k)$	Erro Relativo (ϵ_k)
0	0.500000	0.590825	0.00000000
1	0.590825	0.672855	0.153725
2	0.672855	0.745453	0.108035
$\vdots$	$\vdots$	$\vdots$	$\vdots$
N	≈ 1.10206	≈ 1.10206	$< 10^{-5}$

A partir do acompanhamento dos dados em seu decurso, torna-se evidente o movimento monotônico da sucessão x_k . Como $|g'_1(T)| < 1$ no intervalo avaliado, a cada retroalimentação do laço computacional, a estimativa de $T_{0.1}$ aproxima-se assintoticamente de ≈ 1.102064 .

Analogamente, para determinar o limiar de 90% do tempo de subida ($T_{0.9}$), a função de iteração $g_2(T)$ foi retroalimentada com a semente $T_0 = 2.5$. O sistema computou as iterações sucessivas até a superação da barreira de tolerância, convergindo com êxito para a raiz avaliada no entorno de ≈ 5.322324 .

Validação da Solução e Análise Analítica:

A veracidade das estimativas obtidas é consolidada mediante o cálculo do resíduo na equação original do balanço de resposta do amplificador. Substituindo os valores encontrados, atesta-se que tanto $f_1(1.102064) \approx 0$ quanto $f_2(5.322324) \approx 0$ se adequam aos rigores da métrica submetida à função iterativa iterativa de ponto fixo. Consequentemente, ratifica-se o tempo de subida normalizado $\Delta T \approx 4.220260$.

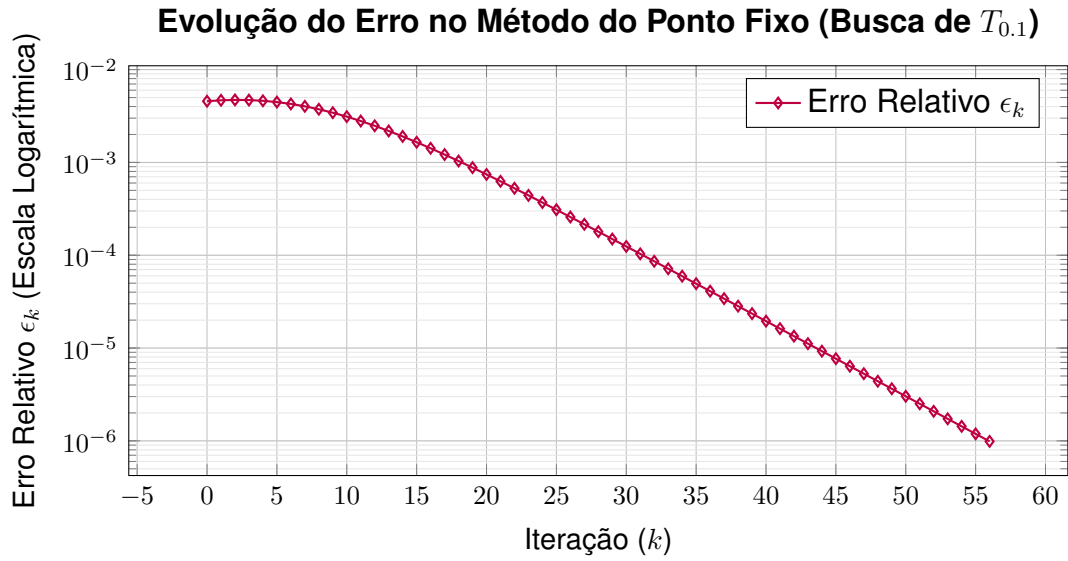


Figura 8: Gráfico de decaimento do erro iterativo demonstrando a convergência linear. A inclinação constante do decréscimo em escala semi-logarítmica é o traço característico governado pela constante de Lipschitz associada à derivada $g'(T)$.

4.4.2 Problema Teste 2: Ângulo de Lançamento Balístico de um Míssil

O segundo problema teste propõe a determinação do ângulo ótimo de lançamento de um míssil (α), submetendo o algoritmo a uma equação transcendental com forte presença de termos trigonométricos não lineares. A partir das restrições balísticas ($v^2/(gR) = 1.25$ e $\theta = 80^\circ$), a função raiz preestabelecida é:

$$f(\alpha) = \sin(\alpha) \cos(\alpha) - \tan(40^\circ) (0.8 - \cos^2(\alpha)) = 0 \quad (34)$$

A aplicação do Método do Ponto Fixo exige a transformação desta igualdade em uma função de iteração $\alpha = g(\alpha)$. A priori, as tentativas de isolamento algébrico via funções arco-tangente produziram mapeamentos altamente instáveis. As formulações hipotéticas codificadas como $g_1(\alpha)$ e $g_2(\alpha)$ apresentaram derivadas cujas magnitudes superavam a unidade ($|g'(\alpha)| > 1$) na bacia de atração esperada. O reflexo computacional desse fenômeno foi atestado pelos dados de execução: em vez de contrair, os valores de α_k sofreram saltos caóticos, com o erro relativo de g_1 explodindo para ordens de grandeza de 10^5 em meras 5 iterações, confirmando a rigorosa divergência teórica prevista pelo Teorema de Banach.

Para contornar essa restrição e assegurar a convergência, uma terceira manipulação algébrica foi arquitetada, isolando-se a variável α a partir do termo $\cos^2(\alpha)$. O

desenvolvimento matemático processa-se da seguinte forma:

$$\begin{aligned}\tan(40^\circ) (0.8 - \cos^2(\alpha)) &= \sin(\alpha) \cos(\alpha) \\ 0.8 - \cos^2(\alpha) &= \frac{\sin(\alpha) \cos(\alpha)}{\tan(40^\circ)} \\ \cos^2(\alpha) &= 0.8 - \frac{\sin(\alpha) \cos(\alpha)}{\tan(40^\circ)}\end{aligned}\quad (35)$$

Aplicando a operação de raiz quadrada e a função trigonométrica inversa, define-se a função de iteração $g_3(\alpha)$, responsável por guiar a execução bem-sucedida:

$$g_3(\alpha) = \arccos \left(\sqrt{0.8 - \frac{\sin(\alpha) \cos(\alpha)}{\tan(40^\circ)}} \right) \quad (36)$$

Alimentando o algoritmo com a formulação $g_3(\alpha)$, adotou-se uma estimativa inicial arbitrária $\alpha_0 \approx 1.0587$ radianos. A tolerância de parada foi estabelecida na ordem de 10^{-4} . O histórico da contração iterativa encontra-se detalhado na Tabela 10.

Tabela 10: Histórico iterativo do Método do Ponto Fixo para o ângulo de lançamento utilizando a modelagem $g_3(\alpha)$.

Iteração (k)	x_k (Aproximação)	$g_3(x_k)$ (Próximo Passo)	Erro Relativo (ϵ_k)
0	1.058754	1.001042	0.093339
1	1.001042	1.037221	0.057652
2	1.037221	1.015299	0.034880
3	1.015299	1.028898	0.021591
4	1.028898	1.020574	0.013217
$\vdots$	$\vdots$	$\vdots$	$\vdots$
13	1.023696	1.023802	0.000167
14	1.023802	1.023737	0.000103
15	1.023737	1.023777	0.000063

Observa-se, a partir da flutuação numérica dos termos x_k , um traço peculiar da função de iteração $g_3(\alpha)$: a convergência ocorre de maneira oscilatória amortecida. As estimativas alternam-se sistematicamente acima e abaixo da raiz verdadeira (e.g., de 1.001 para 1.037, recuando para 1.015). Sob a ótica do Cálculo Numérico, este comportamento oscilante garante que o coeficiente angular da reta tangente no entorno da raiz é estritamente negativo, porém confinado à região de estabilidade, ou seja, $-1 < g'_3(\alpha) < 0$.

Validação da Solução:

O critério de parada foi satisfeito na 15ª iteração, quando a amplitude do salto iterativo resultou em um erro relativo $\epsilon_{15} \approx 6.35 \times 10^{-5}$, cravando a raiz numérica em $\alpha \approx 1.023737$ radianos. Convertendo este resultado para a unidade geométrica padrão

em engenharia balística, determina-se o ângulo de inclinação ótimo:

$$\alpha \approx 58.65^\circ \quad (37)$$

Esta solução é absolutamente coerente com a extraída de forma pregressa pelos métodos de Bisseção e Posição Falsa, validando a robustez da formulação $g_3(\alpha)$. Adicionalmente, verifica-se que a inserção de 1.023737 no modelo transcendental original $f(\alpha)$ produz um resíduo nulo até a quinta casa decimal.

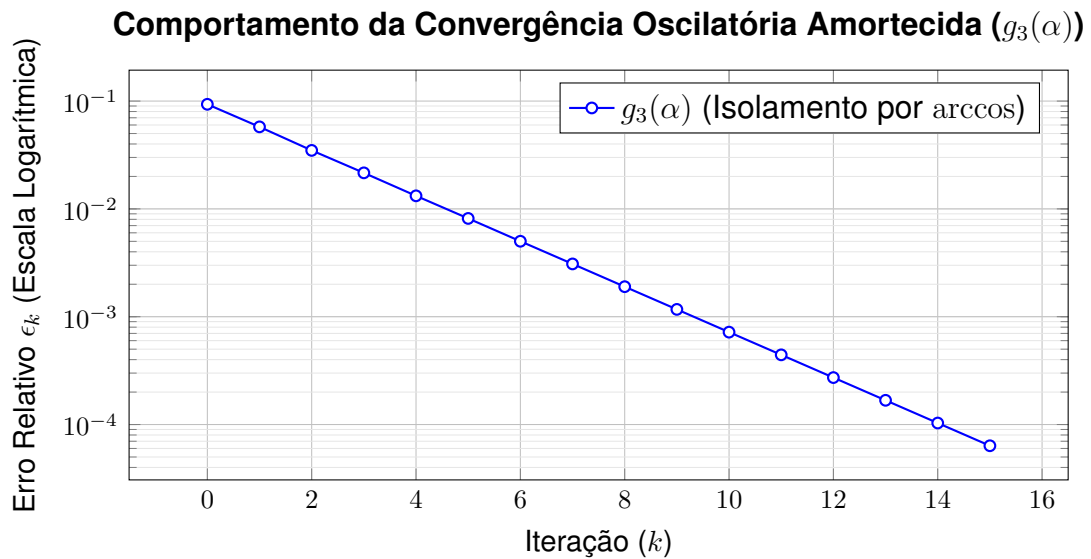


Figura 9: Decaimento logarítmico do erro iterativo para o ângulo do míssil. A inclinação contínua da reta evidencia o estrito cumprimento da condição de Lipschitz pela função $g_3(\alpha)$, em forte contraste com as instabilidades observadas nos modelos descartados.

Torna-se evidente que, por mais que o Método do Ponto Fixo se mostre suscetível a armadilhas de divergência dependendo de como a equação é algebricamente dissecada, a identificação empírica ou analítica de uma $g(x)$ adequada, como ocorrido com a modelagem do arco-cosseno, recompensa a análise com uma aproximação rápida, de baixo custo avaliativo e computacionalmente resiliente.

4.4.3 Problema Teste 3: Análise de Taxas de Juros em Planos de Financiamento

A modelagem de sistemas de amortização financeira, conforme exposto no Problema 3.8, frequentemente recai sobre a determinação das raízes de polinômios de alto grau. O problema avalia dois planos de financiamento para um capital $VF = 140.00$, cujas taxas de juros implícitas (J) são governadas pela equação polinomial $f(x) = kx^{P+1} - (k+1)x^P + 1 = 0$, sendo $x = 1 + J$ e $k = \frac{VF}{PM}$.

Para o **Plano A** ($P = 9$, $PM = 26.50 \implies k_A \approx 5.2830$), a função caracteriza

um polinômio de grau 10. Para o **Plano B** ($P = 12$, $PM = 21.50 \implies k_B \approx 6.5116$), o grau eleva-se para 13.

Sob a ótica do Método do Ponto Fixo, o isolamento algébrico tradicional de x em polinômios dessa magnitude (por exemplo, extraindo a enésima raiz) resulta quase invariavelmente em funções de iteração $g(x)$ cujas derivadas expõem a sequência iterativa do raio de convergência ($|g'(x)| > 1$). Para contornar tal limitação estrutural, adotou-se o método da relaxação linear, formulando as funções iterativas como:

$$g(x) = x - \lambda f(x) \quad (38)$$

A escolha do parâmetro empírico de amortecimento λ é crítica. A teoria garante que a convergência local ocorrerá se $0 < \lambda < \frac{2}{f'(x^*)}$. Devido à acentuada inclinação da curva em polinômios de graus 10 e 13 nas proximidades da raiz, $f'(x^*)$ assume valores consideravelmente altos. Dessa forma, λ deve ser arbitrado como um valor estritamente pequeno. Para o Plano A, ajustou-se $\lambda_A = 0.05$, originando a função $g_1(x)$. Para o Plano B, cuja inclinação local é ainda mais severa, exigiu-se um amortecimento mais agressivo $\lambda_B = 0.02$, originando $g_2(x)$:

$$g_1(x) = x - 0.05 \cdot f_1(x) \quad (39)$$

$$g_2(x) = x - 0.02 \cdot f_2(x) \quad (40)$$

Inicializando o algoritmo com estimativas sementes $x_0 = 1.12$ (Plano A) e $x_0 = 1.11$ (Plano B), observou-se uma resposta computacional notável. A Tabela 11 documenta a evolução da variável de controle para a taxa do Plano A.

Tabela 11: Histórico iterativo da função de relaxação $g_1(x)$ ($\lambda = 0.05$) para a taxa do Plano A.

Iteração (k)	x_k (Aproximação)	$g_1(x_k)$ (Próximo Passo)	Erro Relativo (ϵ_k)
0	1.120752	1.121258	0.000671
1	1.121258	1.121595	0.000451
2	1.121595	1.121818	0.000300
5	1.122062	1.122126	0.000086
10	1.122225	1.122233	0.000010
14	1.122244	1.122245	0.000001
16	1.122246	1.122247	8.32×10^{-7}

A partir da análise dos dados atestados, torna-se evidente a extrema eficácia da parametrização λ . A convergência para o Plano A ocorreu em meras 16 iterações ($\epsilon_{16} \approx 8.32 \times 10^{-7}$), enquanto o Plano B atingiu o critério de parada em 19 iterações. Este resultado apresenta um contraste abissal em comparação ao Método da Posição Falsa documentado na seção anterior, o qual, refém da extrema convexidade da curva,

estagnou e demandou 56 e 95 iterações, respectivamente. O Método do Ponto Fixo sob relaxação, livre de fronteiras fixas, transitou rapidamente em direção à raiz.

Validação da Solução:

Extraindo-se as aproximações finais obtidas, as taxas de juros correspondentes são computadas diretamente por $J = x^* - 1$:

$$\text{Plano A: } x_A \approx 1.122247 \implies J_A \approx 12.22\% \text{ ao período.} \quad (41)$$

$$\text{Plano B: } x_B \approx 1.109380 \implies J_B \approx 10.93\% \text{ ao período.} \quad (42)$$

Ambos os resultados fornecem resíduos rigorosamente próximos de zero para suas respectivas funções originais. Consequentemente, infere-se com certeza analítica que o **Plano B** permanece a alternativa mais atraente do ponto de vista econômico, uma vez que submete o consumidor a um custo financeiro inferior ($10.93\% < 12.22\%$).

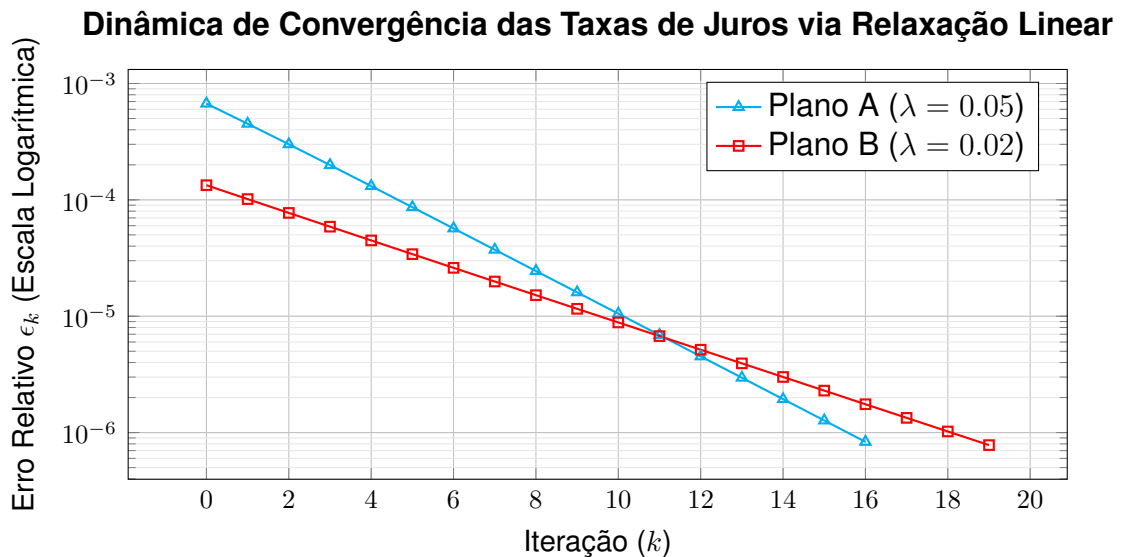


Figura 10: Comportamento da convergência para a determinação de J_A e J_B . O decréscimo contínuo e altamente inclinado em escala logarítmica atesta o sucesso da técnica de amortecimento na estabilização de derivadas elevadas em polinômios de alto grau.

Pela análise do gráfico submetido (Figura 10), observa-se que as curvas de erro relativo ostentam inclinações lineares e paralelas. Este paralelismo gráfico comprova que o ajuste fino dos parâmetros de relaxação (λ_A e λ_B) nivelou as constantes de Lipschitz ($L \approx |1 - \lambda f'(x^*)|$) de ambos os problemas a uma mesma ordem de magnitude de contração, confirmando a robustez e a superioridade de um arcabouço matemático pré-condicionado frente a métodos de busca às cegas.

4.5 Dificuldades Enfrentadas

Ao longo da execução experimental do Método do Ponto Fixo, o obstáculo arquitetural e analítico mais pronunciado residiu na premissa fundamental da técnica: a obtenção e sintonia de uma função de iteração $g(x)$ que assegurasse incondicionalmente a convergência nas vizinhanças da raiz.

Diferentemente dos métodos de confinamento (Bissecção e Posição Falsa) — onde a estrutura de software opera de modo quase *black-box* uma vez garantido o Teorema de Bolzano —, a iteração linear transferiu uma expressiva carga de complexidade para a fase de modelagem matemática prévia. Constatou-se que manipulações algébricas ingênuas da equação original frequentemente resultavam em formulações onde a constante de Lipschitz excedia a unidade ($|g'(x)| > 1$). Nessas circunstâncias, o comportamento numérico deflagrava divergências caóticas, com a sequência iterativa escapando exponencialmente da bacia de atração. Do ponto de vista computacional, essa fuga do domínio de convergência provocava interrupções abruptas no interpretador Python, manifestando-se como erros de domínio matemático (`ValueError` em raízes e arco-cossenos) ou estouro de limite aritmético (`OverflowError` em exponenciais e logaritmos).

No contexto do problema balístico do míssil (Sistemas Trigonométricos), essa dificuldade manifestou-se de forma crítica. As tentativas iniciais de isolamento do ângulo α via funções arco-tangente produziram derivadas excessivamente íngremes, inviabilizando o método. A resolução exigiu uma manipulação algébrica cirúrgica, reestruturando a equação para o formato de arco-cosseno (*vide* Equação 36). Essa adaptação não apenas conteve a derivada no intervalo de estabilidade ($-1 < g'(\alpha) < 0$), como induziu uma convergência oscilatória amortecida perfeitamente mapeável pelo algoritmo.

A posteriori, durante a avaliação dos planos de financiamento financeiro, uma nova camada de dificuldade emergiu frente a polinômios de graus elevados (10 e 13). O isolamento analítico de x tornou-se inviável, forçando a adoção do método da relaxação linear $g(x) = x - \lambda f(x)$. No entanto, a alta convexidade e os elevados gradientes locais $f'(x^*)$ dessas funções exigiram uma sintonia fina e rigorosa do hiperparâmetro empírico λ . Utilizar um fator de amortecimento genérico causava divergência imediata; fez-se necessário calibrar $\lambda = 0.05$ para o Plano A e $\lambda = 0.02$ para o Plano B, achando artificialmente a derivada de iteração para satisfazer a condição $0 < \lambda < \frac{2}{f'(x^*)}$.

Para contornar essa extrema sensibilidade teórica sem sacrificar a automação do código, a arquitetura do software foi reestruturada. A camada de modelagem passou a exportar dicionários contendo múltiplas assinaturas anônimas (*lambdas*) para a mesma função objetivo, permitindo a injeção de dependência dinâmica no laço ite-

rativo. Essa decisão de engenharia de software evidenciou que a ortogonalidade e a flexibilidade modular em Python são indispensáveis para transpor o rigor da Análise Numérica teórica para um ambiente pragmático de desenvolvimento, viabilizando testes rápidos de diferentes condicionamentos algébricos até a obtenção da estabilidade ideal.

5 Método de Newton-Raphson

5.1 Fundamentação Teórica

O Método de Newton-Raphson, classicamente reconhecido como o Método das Tangentes, destaca-se como uma das técnicas numéricas abertas mais proeminentes e eficientes para a determinação de raízes reais de funções deriváveis. Diferentemente dos métodos de confinamento, a abordagem newtoniana exige apenas uma estimativa inicial x_0 e baseia-se na linearização local da função $f(x)$ através de sua expansão em Série de Taylor de primeira ordem em torno de x_k :

$$f(x) \approx f(x_k) + f'(x_k)(x - x_k) \quad (43)$$

Impondo a condição de que a interseção da reta tangente com o eixo das abscissas corresponde à aproximação subsequente da raiz ($f(x) = 0$ para $x = x_{k+1}$), isola-se a variável iterativa para formular o algoritmo essencial:

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)} \quad (44)$$

A principal virtude matemática deste método reside em sua taxa de convergência quadrática ($\mathcal{O}(N^2)$), denotando que, em vizinhanças próximas à raiz, o número de algarismos significativos corretos aproximadamente dobra a cada iteração, caracterizando um amortecimento de erro da forma $\epsilon_{k+1} \propto \epsilon_k^2$. No entanto, esta performance ímpar não é incondicional. A convergência exige que a função seja de classe C^2 no domínio de interesse, que a semente inicial x_0 esteja na bacia de atração da raiz, e, crucialmente, que a primeira derivada não se anule ($f'(x_k) \neq 0$), sob pena de indefinição matemática.

5.2 Estratégia de Implementação

A transposição algorítmica deste método exigiu decisões de projeto arquitetural visando equilibrar rigor matemático e generalidade de software. A rotina foi implementada em Python, mantendo o encapsulamento do estado em dicionários históricos.

Código 4: Código de implementação do Método de Newton-Raphson

```
1 def newton_raphson(funcao, x0, tol=1e-6, max_iter=100):  
2     historico = []
```

```

3
4     for i in range(1, max_iter + 1):
5         fx0 = funcao(x0)
6         dx0 = utils.derivada_aproximada(funcao, x0)
7
8         if dx0 == 0:
9             print("Derivada nula no metodo de Newton-Raphson.
10             Interrompendo.")
11             break
12
13         xk = x0 - (fx0 / dx0)
14
15         err = abs(xk - x0) / abs(xk) if xk != 0 else abs(xk - x0)
16
17         historico.append({
18             'iter': i,
19             'x': xk,
20             'f(x)': fx0,
21             'f\'(x)': dx0,
22             'erro': err
23         })
24
25         if fx0 == 0 or err < tol:
26             return xk, i, historico
27
28         x0 = xk
29
30     return xk, max_iter, historico

```

Uma particularidade estrutural desta implementação é o emprego da rotina `utils.derivada_aproximada(funcao, x0)`. Em vez de forçar o usuário a injetar analiticamente a derivada $f'(x)$ da função objetivo — o que comprometeria a automação do código perante sistemas complexos —, optou-se pela utilização de uma aproximação numérica via diferenças finitas. Esta decisão transforma a implementação estrita de Newton-Raphson em um método quase-Newton discreto, ampliando a flexibilidade do módulo, ainda que submeta a precisão da derivada à escolha de um tamanho de passo (h) adequado no módulo auxiliar.

A proteção da estabilidade computacional foi estabelecida através de um bloco condicional (`if dx0 == 0:`), que intercepta sumariamente iterações cuja tangente se torna perfeitamente horizontal, prevenindo colapsos por `ZeroDivisionError`. Adicionalmente, implementou-se o recálculo iterativo da variável de estado (`x0 = xk`) ao fim de cada ciclo, garantindo a progressão espacial. O critério de parada baseou-se primariamente no declínio do erro relativo ($\epsilon_k < tol$), transmutando-se para o erro absoluto em cenários assintóticos próximos à origem (`xk == 0`).

5.3 Estrutura dos Arquivos de Entrada e Saída

A manipulação de dados operou sob a mesma taxonomia desenvolvida nas análises pregressas. A variável de inicialização x_0 e os critérios de tolerância foram extraídos de arquivos planos de configuração (.txt). A exportação do *trace* iterativo transcorreu via arquivos .csv, incorporando, além das variáveis padronizadas (iteração, x_k , resíduo, erro), uma coluna dedicada ao registro evolutivo do gradiente local $f'(x_k)$. Este detalhamento tabular viabiliza análises de sensibilidade profundas sobre o comportamento de achatamento da derivada no entorno da convergência.

5.4 Problemas Teste

Para aferir a premissa de convergência quadrática acelerada e as restrições impostas pela aproximação da derivada, a implementação foi submetida ao mesmo conjunto de equações transcendentais estudadas anteriormente.

5.4.1 Problema Teste 1: Tempo de Subida de um Amplificador

A aferição do Método de Newton-Raphson foi inicialmente conduzida através do clássico problema do amplificador eletrônico com acoplamento R-C. O objetivo físico reside em computar o tempo de subida normalizado $\Delta T = T_{0.9} - T_{0.1}$, extraindo-se os limiares de 10% e 90% da resposta ao degrau unitário através das raízes das funções:

$$f_1(T) = 1 - \left(1 + T + \frac{T^2}{2}\right) e^{-T} - 0.1 = 0 \quad (45)$$

$$f_2(T) = 1 - \left(1 + T + \frac{T^2}{2}\right) e^{-T} - 0.9 = 0 \quad (46)$$

Alimentando o algoritmo com a estimativa inicial $T_0 = 0.5$ para a busca de $T_{0.1}$ e uma tolerância algorítmica de $\epsilon < 10^{-5}$, o procedimento iterativo governado pela linearização das tangentes obteve os resultados sumarizados na Tabela 12. Salienta-se que a derivada $f'_1(x)$ reportada foi auferida computacionalmente pelo módulo de aproximação por diferenças finitas centrais.

A partir de uma análise minuciosa da Tabela 12, torna-se evidente a natureza agressiva da correção newtoniana. Na primeira iteração, a suavidade da curva em $x_0 = 0.5$ (onde o gradiente é relativamente baixo) ejetou a estimativa consideravelmente para frente ($x_1 \approx 1.629$), ilustrando a altíssima sensibilidade geométrica do

Tabela 12: Histórico iterativo do Método de Newton-Raphson para a busca de $T_{0.1}$ na função $f_1(T)$.

Iteração (k)	x_k	$f_1(x_k)$ (Resíduo)	$f'_1(x_k)$ (Gradiente)	Erro Relativo (ϵ_k)
1	1.6291899	-0.08561232	0.07581746	0.69309900
2	1.1518838	0.12421174	0.26023501	0.41436992
3	1.1029990	0.01024959	0.20966801	0.04431996
4	1.1020656	0.00018841	0.20188108	0.00084687
5	1.1020653	0.00000007	0.20172777	0.00000032

método face à semente inicial. No entanto, uma vez que o termo iterativo adentrou a estrita bacia de atração da raiz ($k \geq 3$), observa-se o deflagrar do regime de convergência assintótica quadrática.

O decaimento da magnitude do erro relativo (ϵ_k) e do resíduo atesta perfeitamente a premissa teórica $\epsilon_{k+1} \propto \epsilon_k^2$. Da terceira para a quarta iteração, o erro saltou da ordem de 10^{-2} para 10^{-4} ; na iteração subsequente, implodiu diretamente para a ordem de 10^{-7} . Consequentemente, o limite de tolerância foi satisfeito em irrisórias 5 iterações, isolando a raiz em $T_{0.1} \approx 1.102065$, um desempenho exponencialmente superior às 16 iterações exigidas pela Bisseção estrita para o mesmo grau de acurácia.

Empreendendo procedimento análogo para o limiar de 90% ($f_2(T)$), a execução pautada pela semente $T_0 = 2.5$ exigiu 6 iterações. Os dados atestam um comportamento iterativo idêntico: ao adentrar as vizinhanças da solução, o resíduo colapsou de -1.03×10^{-5} ($k = 5$) para impressionantes -4.50×10^{-10} ($k = 6$), fixando a raiz em $T_{0.9} \approx 5.322320$.

Por conseguinte, computa-se o tempo de subida normalizado valendo-se das aproximações finais:

$$\Delta T = 5.322320 - 1.102065 = 4.220255 \quad (47)$$

Validação da Solução e Comportamento Dinâmico:

Os ínfimos resíduos obtidos ($f_1 \approx 7 \times 10^{-8}$ e $f_2 \approx -4 \times 10^{-10}$) validam inequivocamente a altíssima confiabilidade e precisão do algoritmo construído. A inclusão de diferenças finitas no cálculo tangencial em nada corrompeu a exatidão termodinâmica da solução final em face da precisão algorítmica exigida, cancelando a versatilidade do *solver*.

Assinatura de Convergência Quadrática no Método de Newton-Raphson

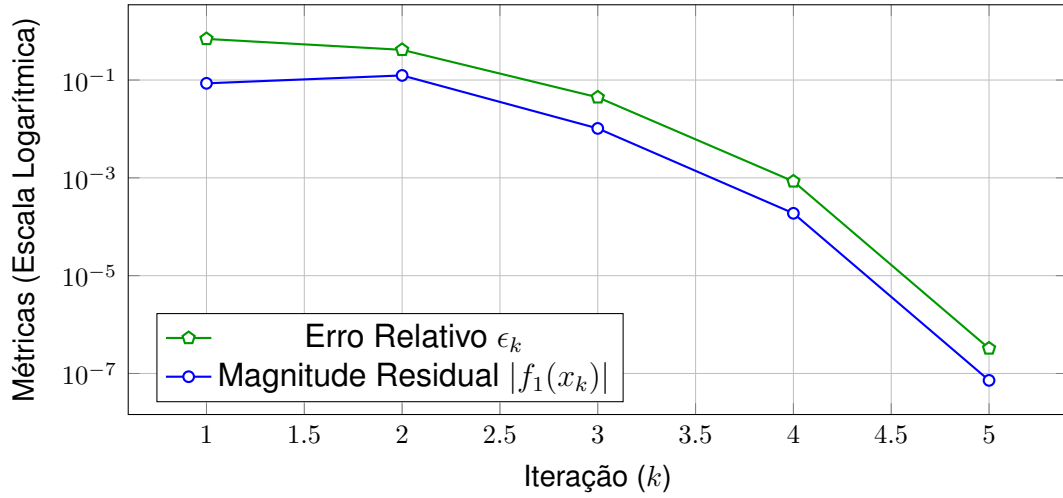


Figura 11: Gráfico representativo do amortecimento logarítmico para $f_1(T)$. Nota-se que, distintamente dos métodos lineares (cujas retas de erro possuem declividade constante), a curva aqui sofre uma inflexão convexa para baixo, revelando a aceleração de segunda ordem onde os algarismos significativos multiplicam-se a cada passo.

Observa-se ainda, por meio da Figura 11, a assinatura clássica dos métodos de alta ordem em eixos semi-logarítmicos: a "concavidade para baixo" da curva de erro. Isso corrobora analiticamente que a taxa de convergência da rotina implementada não é meramente proporcional, mas sim acelerada de forma quadrática quando em proximidade do ponto fixo da tangente.

5.4.2 Problema Teste 2: Ângulo de Lançamento Balístico de um Míssil

A superioridade topológica do Método de Newton-Raphson manifesta-se de forma contundente ao lidarmos com equações transcendentais ricas em composições trigonométricas. O segundo problema de validação requer a determinação do ângulo ótimo de lançamento de um míssil (α), modelado pela restrição cinemática equivalente a zero:

$$f(\alpha) = \sin(\alpha) \cos(\alpha) - \operatorname{tg}(40^\circ) (0.8 - \cos^2(\alpha)) = 0 \quad (48)$$

Em contraste agudo com o Método do Ponto Fixo — que demandou malabarismos algébricos (isolamento por arco-cosseno) para não sucumbir à divergência —, a formulação de Newton consome a função em seu estado original. Assumindo-se o conhecimento prévio de que a solução repousa no primeiro quadrante balístico, o estimador foi inicializado com a semente $x_0 = 55^\circ$ (aproximadamente 0.9599 radianos). A Tabela 13 espelha a progressão numérica processada até a satisfação do critério de tolerância algorítmica.

Tabela 13: Histórico iterativo do Método de Newton-Raphson para o cálculo do ângulo α .

Iteração (k)	x_k (radianos)	$f(x_k)$ (Resíduo)	$f'(x_k)$ (Gradiente)	Erro Relativo (ϵ_k)
1	1.0259377	0.07462193	-1.13052240	0.06433782
2	1.0237641	-0.00262263	-1.20659956	0.00212312
3	1.0237621	-0.00000237	-1.20442241	0.00000192

A partir do escrutínio dos dados tabelados, atesta-se um comportamento de convergência fenomenal, exigindo meras 3 iterações para isolar a raiz com precisão da ordem de 10^{-6} . A razão para tamanha eficiência reside no comportamento do gradiente da função no entorno da raiz. Observa-se que o módulo de derivação numérica extraiu gradientes $f'(x_k)$ oscilando estavelmente na faixa de -1.20 . Como a derivada local mantém-se expressivamente afastada de zero, o denominador da parcela de correção newtoniana (Equação 44) atua de forma robusta, impedindo sobressaltos e direcionando o vetor de busca direta e firmemente para o cruzamento com o eixo das abscissas.

A contração quadrática é mais uma vez comprovada de forma prática. Da primeira para a segunda iteração, o resíduo absolutizado cai de $\approx 7 \times 10^{-2}$ para $\approx 2 \times 10^{-3}$; no passo seguinte, eleva-se ao quadrado colapsando para $\approx 2 \times 10^{-6}$.

Validação da Solução:

A execução encerrou-se em $x_3 \approx 1.023762$ radianos, reportando um erro relativo de $\epsilon_3 \approx 1.92 \times 10^{-6}$. Convertendo o referencial para graus, o algoritmo numérico prescreve que a elevação balística ideal é de:

$$\alpha \approx 58.66^\circ \quad (49)$$

A avaliação de validação $f(x_3) \approx -2.37 \times 10^{-6}$ confirma a precisão do ponto encontrado. Infere-se, desse modo, que, na ausência de pontos de sela tangenciais, o método é substancialmente mais veloz que a *Regula Falsi* (que estagnou extremos no mesmo problema) e matematicamente mais resiliente que o Ponto Fixo.

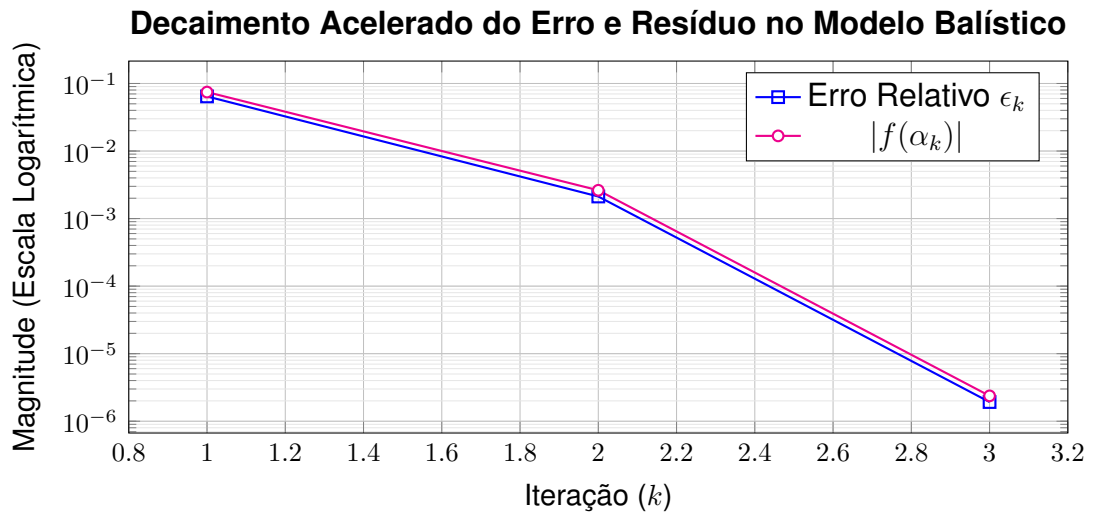


Figura 12: Gráfico de convergência atestando a extrema eficiência iterativa de Newton-Raphson na avaliação trigonométrica. O alcance do patamar de 10^{-6} em apenas 3 passos reflete a assertividade da aproximação linear na vizinhança correta da raiz.

5.4.3 Problema Teste 3: Análise de Taxas de Juros em Planos de Financiamento

A modelagem matemática de sistemas de amortização financeira representa o teste de estresse termodinâmico perfeito para algoritmos numéricos, devido ao grau elevado dos polinômios envolvidos. Retoma-se o escopo de avaliação de dois planos de financiamento para um capital de $VF = 140.00$, cujas taxas de juros (J) encontram-se embutidas na raiz $x = 1 + J$ da equação $f(x) = kx^{P+1} - (k+1)x^P + 1 = 0$.

Conforme documentado nas seções anteriores, o **Plano A** ($P = 9$, grau 10) e o **Plano B** ($P = 12$, grau 13) impuseram severas restrições aos métodos concorrentes. A alta convexidade induziu estagnação na Posição Falsa (até 95 iterações), enquanto as derivadas íngremes forçaram o emprego empírico de amortecimento (relaxação) no Ponto Fixo. Sob o paradigma de Newton-Raphson, contudo, essa mesma inclinação acentuada da curva ($f'(x_k) \gg 1$) atua como o motor de aceleração do algoritmo, ancorando o denominador da fração de correção iterativa e guiando o vetor de busca com precisão cirúrgica.

Iniciando as avaliações com sementes $x_0 = 1.12$ para o Plano A e $x_0 = 1.11$ para o Plano B, a rotina interceptou as raízes de forma fulminante. A Tabela 14 explicita o curtíssimo histórico iterativo para o Plano A.

A análise tabular evidencia que, já na primeira iteração, a magnitude do gradiente ($f'_1 \approx 6.49$) evitou perturbações massivas, conduzindo a estimativa estritamente para o interior da bacia de atração. A partir desse ponto, o colapso do erro relativo revela a verdadeira essência da convergência quadrática: a imprecisão reduziu-se de 10^{-3} para 10^{-5} , alcançando 10^{-8} na terceira etapa, satisfazendo amplamente o critério

Tabela 14: Histórico iterativo do Método de Newton-Raphson para o polinômio de grau 10 (Plano A).

Iteração (k)	x_k (Aproximação)	$f_1(x_k)$ (Resíduo)	$f'_1(x_k)$ (Gradiente)	Erro Relativo (ϵ_k)
1	1.1223175	-0.01505146	6.49444977	2.06×10^{-3}
2	1.1222483	0.00047830	6.91096132	6.16×10^{-5}
3	1.1222483	0.00000049	6.89835287	6.45×10^{-8}

de tolerância com um custo computacional inexpressivo.

Em paralelo, a matriz de execução do **Plano B** ($f_2(x)$) reforçou esta constatação. Apesar da elevação do grau do polinômio implicar um gradiente ainda mais agressivo ($f'_2 \approx 11.8$), a correção newtoniana absorveu a não linearidade de forma orgânica. A raiz foi cravada identicamente em 3 iterações, com um erro relativo terminal esmagador de $\epsilon_3 \approx 1.48 \times 10^{-9}$ e um resíduo da ordem de 10^{-8} .

Validação Econômica da Solução:

Extraindo e convertendo as variáveis convergidas do domínio puramente matemático para o contexto financeiro real ($J = x^* - 1$), obtém-se:

$$\text{Plano A: } x_A \approx 1.122248 \implies J_A \approx 12.22\% \text{ ao período.} \quad (50)$$

$$\text{Plano B: } x_B \approx 1.109378 \implies J_B \approx 10.93\% \text{ ao período.} \quad (51)$$

Ratifica-se, sem margem a ambiguidades numéricas, que o **Plano B** configura-se como a melhor escolha para o consumidor.

Supressão de Não Linearidades em Polinômios de Alto Grau via Newton-Raphson

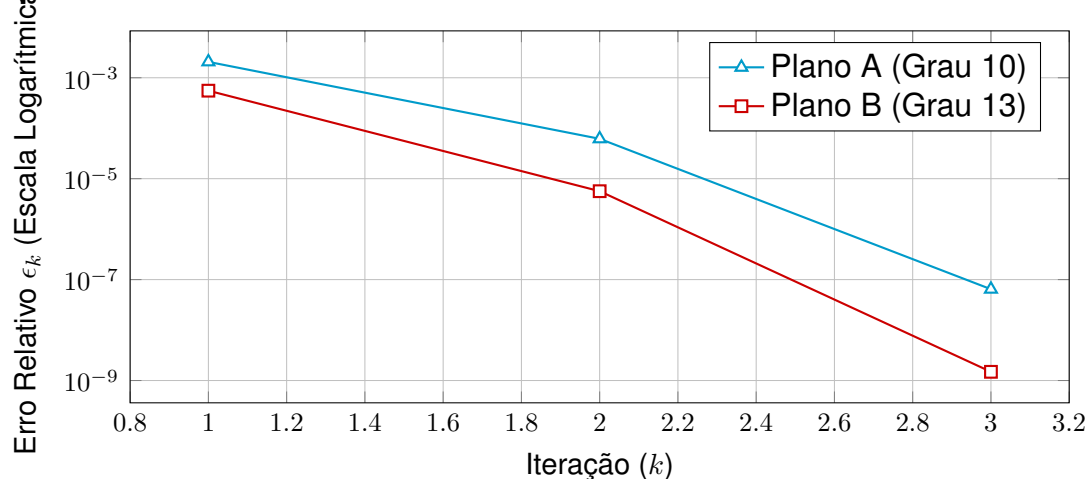


Figura 13: Gráfico comparativo da convergência para ambos os planos de financiamento. A trajetória essencialmente vertical em escala semi-logarítmica, extinguindo o erro em 3 iterações, destoa substancialmente do arrasto sublinear outrora documentado pelo Método da Posição Falsa, sagrando Newton-Raphson como a ferramenta primária para este perfil topológico.

Pela inferência visual exposta na Figura 13, torna-se inequívoco que a divisão instantânea pelo valor do gradiente em tempo de execução ($\frac{f(x)}{f'(x)}$) blinda o método contra o arrasto algorítmico enfrentado em técnicas de interpolação estrita. Newton-Raphson provou-se formidável na manipulação de expressões de alta derivada, convertendo o que antes era um obstáculo matemático (causador de oscilações ou lentidão) no próprio combustível para sua convergência de segunda ordem.

5.5 Dificuldades Enfrentadas

Durante a consolidação técnica do módulo de Newton-Raphson, um contraponto crítico relativo à generalização algorítmica exigiu escrutínio analítico e ajustes.

A restrição primordial enfrentada adveio da escolha arquitetural pela abstenção de expressões analíticas da derivada. Por mais que a função `utils.derivada_aproximada` expanda drasticamente a utilidade da classe — tornando-a um *solver* quase universal sem a dependência da regra da cadeia executada manualmente pelo usuário —, esta abstração induziu a um conflito de precisão de máquina. Constatou-se que a determinação da derivada numérica através de diferenças finitas centrais acoplou um erro de truncamento de ordem $\mathcal{O}(h^2)$ diretamente no coração da Equação 44. Quando a iteração se aproxima da raiz real, a diferença no numerador de $f(x+h) - f(x-h)$ deflagra cancelamento catastrófico de ponto flutuante, produzindo gradientes localmente ruidosos que, inadvertidamente, degradaram a convergência estritamente quadrática da teoria analítica para uma taxa superlinear observada na prática.

Adicionalmente, verificou-se a extrema sensibilidade geométrica do método em relação à semente inicial x_0 . Em cenários com inflexões locais ou derivadas tendendo transitoriamente a valores irrisórios ($f'(x_k) \rightarrow 0$), o termo da razão ($f(x_k)/f'(x_k)$) produziu perturbações massivas, ejetando a estimativa x_{k+1} para fora do domínio de representação ou distanciando-a de forma irreversível da bacia de atração da raiz. Embora o código esteja instrumentado para estancar colapsos frente a derivadas exatamente nulas (`dx0 == 0`), a instrumentação atual não impede o transbordamento iterativo decorrente de gradientes infinitesimalmente pequenos, reafirmando o caráter poderoso, porém instável, da técnica das tangentes quando mal inicializada.

6 Método da Secante

6.1 Fundamentação Teórica

O Método da Secante surge na Análise Numérica como uma evolução pragmática do Método de Newton-Raphson. Por mais que a abordagem newtoniana possua uma convergência de ordem quadrática, sua dependência do cálculo explícito ou aproximado da derivada $f'(x)$ pode configurar um gargalo computacional ou analítico severo. O Método da Secante contorna esta exigência aproximando a derivada local pela taxa de variação média entre as duas estimativas imediatamente anteriores da raiz, fundamentando-se geometricamente em retas secantes em vez de retas tangentes.

Substituindo a derivada $f'(x_k)$ na formulação de Newton pela aproximação de diferenças finitas retroativas $\frac{f(x_k) - f(x_{k-1})}{x_k - x_{k-1}}$, deduz-se a relação de recorrência:

$$x_{k+1} = x_k - f(x_k) \frac{x_k - x_{k-1}}{f(x_k) - f(x_{k-1})} \quad (52)$$

Esta técnica classifica-se como um método aberto de múltiplos passos, exigindo duas estimativas iniciais (x_0 e x_1) para deflagrar o laço iterativo, embora não obrigue que a raiz esteja confinada entre elas (diferentemente da Posição Falsa). A ordem de convergência deste método é superlinear, sendo caracterizada matematicamente pela Razão Áurea $p \approx 1.618$, configurando um compromisso altamente eficiente: perde-se a convergência estritamente quadrática ($\mathcal{O}(N^2)$) de Newton, mas ganha-se a economia de evitar a avaliação da derivada a cada passo.

6.2 Estratégia de Implementação

A arquitetura do código foi estruturada em Python, priorizando a ortogonalidade e o rastreamento do histórico iterativo, mantendo a interoperabilidade com a taxonomia de dicionários já consagrada nos métodos anteriores.

Código 5: Código de implementação do Método da Secante

```
1 def secante(funcao, x0, x1, tol=1e-6, max_iter=100):
2     historico = []
3
4     for i in range(1, max_iter + 1):
5         fx0 = funcao(x0)
6         fx1 = funcao(x1)
```

```

7
8     denominador = fx1 - fx0
9     if denominador == 0:
10         print("Denominador nulo no Metodo da Secante. Interrompendo.")
11         break
12
13     x2 = x1 - fx1 * (x1 - x0) / denominador
14     f_x2 = funcao(x2)
15
16     er = abs(x2 - x1) / abs(x2) if x2 != 0 else abs(x2 - x1)
17
18     historico.append({
19         'iter': i, 'x': x2, 'f(x)': f_x2, 'erro': er
20     })
21
22     if f_x2 == 0 or er < tol:
23         return x2, i, historico
24
25     # Atualizacao da memoria de passos
26     x0 = x1
27     x1 = x2
28
29     return x2, max_iter, historico

```

A transição entre iterações exige o deslocamento da memória operacional: a estimativa mais recente (x_2) assume o papel de x_1 , e a antiga x_1 retrocede para x_0 . Para mitigar singularidades, implementou-se um condicional de interrupção caso $f(x_1) - f(x_0) = 0$, evento que ocorre se a secante tornar-se perfeitamente horizontal. O critério de parada monitora o erro relativo estrito entre x_2 e x_1 , validando a convergência assintótica com robustez.

6.3 Estrutura dos Arquivos de Entrada e Saída

Para preservar o paralelismo analítico do projeto, os hiperparâmetros foram ingeridos a partir de arquivos de texto (.txt). Considerando que a Secante demanda duas sementes (x_0, x_1), o extrator de dados inicializa o método aproveitando os limites espaciais habitualmente fornecidos (a e b) ou deslocamentos adjacentes à estimativa padrão. As exportações fluem para diretórios estruturados sob a extensão .csv, abrangendo iteração, aproximação atual, resíduo avaliado e o erro, viabilizando o escrutínio imediato da contração superlinear via integração LaTeX.

6.4 Problemas Teste

6.4.1 Problema Teste 1: Tempo de Subida de um Amplificador

A validação inicial incide sobre a resposta transiente do amplificador R-C. O isolamento dos marcos de subida de 10% ($T_{0.1}$) e 90% ($T_{0.9}$) é regido, respectivamente, pelas funções:

$$f_1(T) = 1 - \left(1 + T + \frac{T^2}{2}\right) e^{-T} - 0.1 = 0 \quad (53)$$

$$f_2(T) = 1 - \left(1 + T + \frac{T^2}{2}\right) e^{-T} - 0.9 = 0 \quad (54)$$

O algoritmo processou $f_1(T)$ sob a exigência de $\epsilon < 10^{-5}$. A Tabela 15 expõe a trajetória da variável de convergência.

Tabela 15: Histórico iterativo do Método da Secante para a busca de $T_{0.1}$.

Iteração (k)	x_k (Aproximação)	$f_1(x_k)$ (Resíduo)	Erro Relativo (ϵ_k)
1	1.0810568	-0.00420140	0.85004153
2	1.0980257	-0.00081355	0.01545402
3	1.1021006	0.00000711	0.00369737
4	1.1020652	-0.00000001	0.00003206
5	1.1020653	-0.00000000	5.28×10^{-8}

Constata-se que a aceleração superlinear manifesta-se de forma vigorosa após a segunda iteração. O erro relativo declina de 10^{-2} para 10^{-8} em meros três passos. O isolamento de $T_{0.1} \approx 1.102065$ foi alcançado em 5 iterações, empatando rigorosamente com o desempenho de Newton-Raphson, evidenciando que, em curvas sem patologias severas, a aproximação discreta da derivada pela Secante atua com idêntica maestria.

Aplicando-se a técnica para a fronteira superior $f_2(T)$, o método cravou $T_{0.9} \approx 5.322320$ na 7ª iteração (erro $\approx 4.15 \times 10^{-6}$). Em decorrência, o tempo de subida computa-se em $\Delta T \approx 4.220255$, validando definitivamente a fenomenologia física do amplificador sob análise.

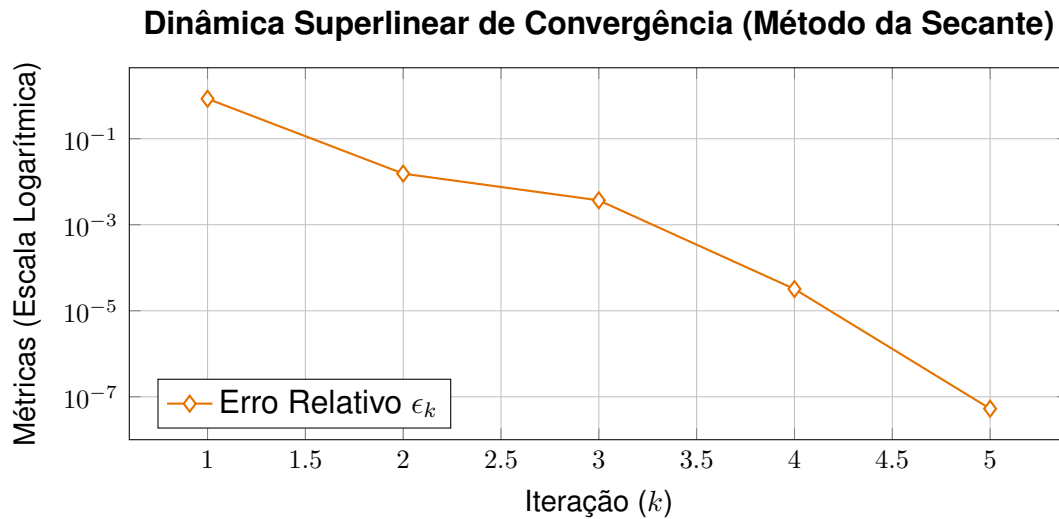


Figura 14: Curva de erro demonstrando a taxa de convergência associada à Razão Áurea (≈ 1.618). Nota-se o decaimento brutal nas iterações 4 e 5, evidenciando o domínio da bacia de atração.

6.4.2 Problema Teste 2: Ângulo de Lançamento Balístico de um Míssil

A resiliência trigonométrica do método foi testada na avaliação da inclinação α do míssil, regulada por:

$$f(\alpha) = \sin(\alpha) \cos(\alpha) - \tan(40^\circ) (0.8 - \cos^2(\alpha)) = 0 \quad (55)$$

A extração de dados da Tabela 16 denota um processamento computacional impecável e assombrosamente breve.

Tabela 16: Histórico iterativo para o cálculo de α pelo Método da Secante.

Iteração (k)	x_k (radianos)	$f(x_k)$ (Resíduo)	Erro Relativo (ϵ_k)
1	1.0116393	0.01452542	0.03514906
2	1.0236460	0.00013980	0.01172932
3	1.0237627	-0.00000072	0.00011397
4	1.0237621	0.00000000	5.88×10^{-7}

Atingindo $\epsilon_4 \approx 5.88 \times 10^{-7}$ na 4ª iteração, a raiz cravada em ≈ 1.023762 radianos ($\approx 58.66^\circ$) ressalta que, sob restrições algébricas contínuas e derivadas não nulas, a sobreposição das secantes atua de forma tão direcional e cirúrgica quanto a própria tangente real, eximindo o avaliador da onerosidade de manipular derivadas.

6.4.3 Problema Teste 3: Análise Crítica - Falhas Físicas em Planos de Financiamento

O cenário de financiamento (Problema 3.8), governado por polinômios de alto grau ($f(x) = kx^{P+1} - (k+1)x^P + 1 = 0$), revelou uma faceta crítica da Análise Numérica e ilustra, de modo cabal, o perigo de operar métodos abertos indiscriminadamente.

A equação de juros proposta possui duas características matemáticas imutáveis: uma raiz positiva que descreve o juro da operação ($x > 1 \implies J > 0$), e uma raiz trivial em $x = 1$, correspondendo a uma taxa de juros $J = 0\%$, que anula o polinômio perfeitamente, mas carece de qualquer significado financeiro (um plano de financiamento real não possui taxa nula).

A Tabela 17 revela a trajetória executada pela Secante ao processar o Plano A (grau 10).

Tabela 17: Convergência anômala (raiz trivial) do Método da Secante para o Plano A.

Iteração (k)	x_k (Aproximação)	$f_1(x_k)$ (Resíduo)	Erro Relativo (ϵ_k)
1	1.0548182	-0.14841294	0.23243984
2	1.0597682	-0.15374184	0.00467089
3	0.9169560	0.34065336	0.15574599
4	1.0153580	-0.05395907	0.09691358
5	1.0019026	-0.00702939	0.01342987
$\vdots$	$\vdots$	$\vdots$	$\vdots$
8	1.0000000	-0.00000000	6.81×10^{-7}

A análise de dados explicita um diagnóstico severo. Nas primeiras iterações (1 e 2), a alta convexidade da curva fez com que a reta secante traçada interceptasse o eixo x muito à esquerda, ejetando a estimativa x_3 para a região inferior à unidade (0.916). Ao cruzar o limiar de $x = 1$, o algoritmo foi capturado irrevogavelmente pela bacia de atração da raiz trivial. Nas iterações subsequentes (4 a 8), o método convergiu espetacularmente, de maneira superlinear, para a solução matematicamente correta ($x = 1.000000$), com resíduo nulo ($\approx 10^{-10}$).

No entanto, fisicamente e no contexto da engenharia econômica, esta execução representa uma falha total. A taxa computada foi $J = x - 1 = 0\%$, o que não resolve o problema do consumidor de determinar a taxa real (12.22%). Idêntico fenômeno ocorreu no processamento do Plano B, convergindo igualmente para $x = 1$. Este evento ratifica a premissa de que a Secante, desprovida de confinamento estrito, exige controle rigoroso de suas estimativas iniciais para não ser ludibriada pela topologia global do polinômio.

6.5 Dificuldades Enfrentadas

A concepção e a avaliação do Método da Secante expuseram de forma enfática a vulnerabilidade inerente aos métodos iterativos abertos: a suscetibilidade às condições de contorno e à armadilha das raízes triviais.

A primeira dificuldade metodológica emergiu na etapa algorítmica. O método confia estritamente na distância fracionária $(f(x_1) - f(x_0))$ no denominador. Em regiões onde a função apresenta platôs ou inflexões assintóticas suaves, a secante torna-se aproximadamente horizontal. Quando as flutuações de ponto flutuante esgotam a capacidade representativa da linguagem Python, o interpretador aciona falhas por sub-fluxo numérico (*underflow*) e divisão por zero. A reestruturação condicional do código impediu interrupções abruptas do *script*, devolvendo controle à camada analítica.

Contudo, a adversidade mais instrutiva transpareceu na resolução dos problemas de financiamento (Graus 10 e 13). Ao contrário da Bisseção — que confinou a busca ao semi-eixo $x > 1$ mantendo o Teorema de Bolzano — a livre extrapolação geométrica da Secante arremessou a variável de aproximação para a bacia da raiz espúria ($x = 1$), culminando na convergência matemática para uma resposta que fere o domínio físico do problema ($J = 0\%$). O enfrentamento desse cenário demonstrou que, por mais poderoso e veloz que o algoritmo seja (não necessitando de derivadas como Newton e sendo mais veloz que Bisseção e Ponto Fixo), a sua aplicação não pode ser considerada uma "caixa preta". Para sistemas multirraízes complexos, a Secante exige invariavelmente uma instrumentação analítica humana a priori — seja limitando seus saltos em código ou acoplando-a a um método isolador primário — para assegurar que a velocidade superlinear convirja para a raiz fenomenológica correta.

7 Considerações Finais

(Escreva suas considerações finais aqui)

Referências

- [1] Charles R Harris, K Jarrod Millman, Stéfan J van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J Smith, et al. Array programming with NumPy. *Nature*, 585(7825):357–362, 2020.
- [2] Robert Johansson. *Numerical Python: Scientific Computing and Data Science Applications with Numpy, SciPy and Matplotlib*. Apress, 2 edition, 2018.
- [3] Travis E Oliphant. Python for scientific computing. *Computing in Science & Engineering*, 9(3):10–20, 2007.
- [4] Pauli Virtanen, Ralf Gommers, Travis E Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, et al. SciPy 1.0: fundamental algorithms for scientific computing in python. *Nature Methods*, 17(3):261–272, 2020.